Riješeni zadaci uz Predavanje 2

Dolje navedeni zadaci po svom sadržaju prate Predavanje 2.

1. Kompleksni brojevi svoju najveću primjenu nalaze u elektrotehnici. Recimo, u teoriji električnih kola uvodi se pojam *impedanse*, koja je kompleksan broj $\bar{Z}$ definiran kao $\bar{Z} = R + X\,i$, gdje je R tzv. aktivni otpor, a X tzv. reaktivni otpor (ili reaktansa), koji može biti i negativan. Ukoliko imamo paralelni spoj n elemenata čije su impedanse $\bar{Z}_1, \bar{Z}_2, \ldots, \bar{Z}_n$, ukupna impedansa paralelne veze $\bar{Z}$ računa se kao $\bar{Z} = 1/(1/\bar{Z}_1 + 1/\bar{Z}_2 + \ldots + 1/\bar{Z}_n)$. Drugim riječima, situacija je vrlo slična paralelnom spajanju otpornika, osim što se ovdje radi sa kompleksnim impedansama. Napisati program u kojem se korisnika traži da se unese broj elemenata n, a zatim n impedansi $\bar{Z}_k$ za sve k od 1 do n. Impedanse se unose kao kompleksni brojevi, tačnije kao parovi realnih i imaginarnih dijelova unutar zagrada, razdvojeni zarezom. Nakon toga, program treba da izračuna i ispiše impedansu paralelne veze svih n elemenata, ponovo kao kompleksan broj. Dijalog između programa i korisnika treba da izgleda poput sljedećeg:

```
Unesite broj elemenata: 3

Z_1 = (3.5,2.8)
Z_2 = (10,-1.54)
Z_3 = (12.37,0.24)

Paralelna veza ovih elemenata ima impedansu Z_ = (2.51479,0.897637).
```

Za realizaciju programa potrebno je koristiti kompleksni tip podataka. Pri tome, nije dozvoljeno nikakve kontejnerske tipove podataka (nizove, vektore, moderne nizove, dekove, itd.) Također, može se pretpostaviti da će svi uneseni podaci biti smisleni.

Rješenje:

Kontejnerske tipove podataka možemo izbjeći tako što ćemo sumu $1/\bar{Z}_1 + 1/\bar{Z}_2 + \ldots + 1/\bar{Z}_n$ računati "u hodu", uporedo sa unosom podataka. Naime, na početku inicijaliziramo promjenljivu koja čuva sumu na nulu, a zatim nakon unosa svakog broja njegovu recipročnu vrijednost prosto nadodamo na sumu. Po okončanju unosa, tražena impedansa je recipročna vrijednost nađene sume. Sâm unos kompleksnih brojeva u traženom obliku jako je olakšan činjenicom da kompleksni tipovi podataka podržavaju unos s ulaznog toka pomoću operatora "`>>`", i to upravo u traženom obliku (kao par brojeva u zagradama, tako da za unos nije potrebno vršiti nikakve dodatne komplikacije). Nakon ove diskusije, sasvim je lako sastaviti traženi program, koji može izgledati poput sljedećeg:

```
#include <iostream>
#include <complex>

int main() {
  int n;
  std::cout << "Unesite broj elemenata: ";
  std::cin >> n;
  std::cout << std::endl;
  std::complex<double> suma, z;
  for(int i = 1; i <= n; i++) {
    std::cout << "Z_" << i << " = ";
    std::cin >> z;
    suma += 1. / z;
  }
  std::cout << "\nParalelna veza ovih elemenata ima impedansu Z_ = "
    << 1. / suma << ".\n";
  return 0;
}
```

U program je uključena biblioteka sa zaglavljem "`complex`", da bismo dobili podrške kompleksnom tipu podataka. Treba samo obratiti pažnju na izraze "`1. / z`" i "`1. / suma`". Prividno ekvivalentni izrazi "`1 / z`" i "`1 / suma`" ne bi radili, jer nije dozvoljeno miješanje cjelobrojnih i kompleksnih tipova u istom izrazu (ne smijemo zaboraviti da je "`1`" konstanta tipa "`int`", dok je "`1.`" tipa "`double`").

2. Prepraviti prethodni program tako što će se umjesto impedansi $\bar{Z}_k$ sa tastature posebno unositi aktivni i reaktivni otpori R_k i X_k, i što će se na kraju ispisivati posebno aktivni i reaktivni otpor paralelne veze elemenata. Dijalog između programa i korisnika treba da izgleda poput sljedećeg:

```
Unesite broj elemenata: 3

R1 = 3.5
X1 = 2.8

R2 = 10
X2 = -1.54

R3 = 12.37
X3 = 0.24

Paralelna veza ovih elemenata ima R = 2.51479 i X = 0.897637.
```

Rješenje:

Računanje impedanse obavlja se na isti način kao u prethodnom programu. Bitna je razlika što se ovdje s tastature unose dva realna broja (aktivni i reaktivni otpor), od kojih formiramo jedan kompleksni broj, koristeći pretvorbu para realnih vrijednosti u jednu kompleksnu. Također, kada na kraju nađemo impedansu paralelne veze, odgovarajući aktivni i reaktivni otpor jednaki su realnom i imaginarnom dijelu nađene impedanse, koje možemo dobiti uz pomoću funkcija "`real`" i "`imag`". Ovo vodi ka programu poput sljedećeg:

```
#include <iostream>
#include <complex>

int main() {
  int n;
  std::cout << "Unesite broj elemenata: ";
  std::cin >> n;
  std::cout << std::endl;
  std::complex<double> suma;
  for(int i = 1; i <= n; i++) {
    double r, x;
    std::cout << "R" << i << " = ";
    std::cin >> r;
    std::cout << "X" << i << " = ";
    std::cin >> x;
    suma += 1. / std::complex<double>(r, x);
    std::cout << std::endl;
  }
  suma = 1. / suma;
  std::cout << "Paralelna veza ovih elemenata ima R = " << suma.real()
    << " i X = " << suma.imag() << ".\n";
  return 0;
}
```

3. Prepravite prethodni program tako što će se sve impedanse $\bar{Z}_k$ umjesto preko aktivnog otpora R_k i i reaktivnog otpora X_k zadavati preko tzv. prividnog otpora Z_k i faznog pomaka φ_k, pri čemu vrijedi $\bar{Z}_k = Z_k\, e^{i\varphi_k}$. Program na kraju treba prikazati prividni otpor i fazni pomak za čitavu paralelnu vezu. Fazni pomak treba zadavati i ispisivati u stepenima. Dijalog između programa i korisnika treba da izgleda poput sljedećeg:

```
Unesite broj elemenata: 2

Z1 = 10.5
fi1 = 30

Z2 = 2.8
fi2 = -47.6

Paralelna veza ovih elemenata ima Z = 2.57147 i fi = -33.7613.
```

Rješenje:

Ovaj zadatak je sličan prethodnom, samo se razlikuje po tome kako se unose impedanse. Umjesto kao aktivni i reaktivni otpor, unosi se prividni otpor Z i fazni pomak φ, nakon čega se impedansa dobija kao $\bar{Z} = Ze^{i\varphi}$. Međutim, upravo to radi funkcija "`polar`". Naime, konstrukcija "`polar(r, φ)`" daje kao rezultat kompleksni broj $re^{i\varphi}$ (odnosno $r(\cos\varphi + i\sin\varphi)$ zapisano u trigonometrijskom obliku). Međutim, kako funkcija "`polar`" očekuje fazu u radijanima, a mi je zadajemo u stepenima, potrebna je pretvorba stepeni u radijane, koju možemo dobiti množenjem s π/180 (znajući da π radijana ima 180 stepeni). Nakon što dobijemo impedansu paralelne veze, odgovarajući prividni otpor i fazni pomak nisu ništa drugo nego modul i argument te impedanse, koje možemo dobiti pomoću funkcija "`abs`" i "`arg`". Pri tome, kako funkcija "`arg`" daje argument (fazu) u radijanima, a mi je želimo u sekundama, obavićemo pretvorbu radijana u sekunde množenjem s 180/π. Vrijednost konstante π najtačnije možemo dobiti koristeći formulu $\pi = 4 \cdot \operatorname{arctg} 1$, pri čemu za računanje arkustangensa možemo koristiti funkciju "`atan`" (strogo uzevši, verzija ove funkcije koja prima realne argumente zahtijeva uključivanje biblioteke s zaglavljem "`cmath`", mada većina kompajlera automatski uključuje ovu biblioteku čim se uključi biblioteka s zaglavljem "`complex`", bez obzira što takvo ponašanje nije propisano standardom). Kako će nam vrijednost ove konstante trebati na više mjesta u programu, definiraćemo imenovanu konstantu nazvanu "`PI`" koja će čuvati njenu vrijednost. Provedena razmatranja vode ka sljedećem programu:

```
#include <iostream>
#include <complex>
#include <cmath>

int main() {
  const double PI = 4 * std::atan(1);
  int n;
  std::cout << "Unesite broj elemenata: ";
  std::cin >> n;
  std::cout << std::endl;
  std::complex<double> suma;
  for(int i = 1; i <= n; i++) {
    double z, fi;
    std::cout << "Z" << i << " = ";
    std::cin >> z;
    std::cout << "fi" << i << " = ";
    std::cin >> fi;
    suma += 1. / std::polar(z, fi * PI / 180);
    std::cout << std::endl;
  }
  suma = 1. / suma;
  std::cout << "Paralelna veza ovih elemenata ima Z = " << std::abs(suma)
    << " i fi = " << std::arg(suma) * 180 / PI << ".\n";
  return 0;
}
```

4. Napisati prvo funkciju "`DaLiJeProst`" sa jednim parametrom "`n`" koji predstavlja neki cijeli broj, a koja kao rezultat vraća logičku vrijednost "tačno" ili "netačno" u ovisnosti od toga da li je parametar prost broj ili ne. Zatim napisati funkciju "`ProstiBrojeviUOpsegu`", također sa dva cjelobrojna parametra "`a`" i "`b`" koja kao rezultat vraća vektor koji se sastoji od svih prostih brojeva u intervalu od "`a`" do "`b`" uključivo. Napisanu funkciju iskoristiti u glavnom programu koji traži da se sa tastature unesu dva cijela broja, a zatim za sve cijele brojeve u opsegu između ta dva unesena broja ispisuje koji su prosti brojevi u datom opsegu, ili informaciju da nema prostih brojeva u datom opsegu. Dijalozi između korisnika i programa trebali bi izgledati poput sljedećih (ovisno ima li ili nema prostih brojeva u zadanom opsegu):

```
Unesite pocetnu i krajnju vrijednost: 15 30
Prosti brojevi u rasponu od 15 do 30 su: 17, 19, 23, 29
```

```
Unesite pocetnu i krajnju vrijednost: 24 28
Nema prostih brojeva u rasponu od 24 do 28!
```

Pretpostavite da će korisnik unijeti smislene podatke, tj. nije potrebno provjeravati da li su zaista uneseni cijeli brojevi. Pri tome, program treba da radi ispravno za sve ulazne podatke koji mogu stati u cjelobrojni tip "`int`".

Rješenje:

Prosti brojevi su oni prirodni brojevi koji imaju tačno dva različita djelioca u skupu $\mathbb{N}$: jedinicu i sâmog sebe. Treba napomenuti da prividno slična definicija po kojoj su prosti brojevi oni i samo oni prirodni brojevi koji su djeljivi samo s jedinicom i sa sâmim sobom nije u potpunosti tačna. Naime, prema toj definiciji broj 1 bi bio prost broj, jer je zaista djeljiv samo s jedinicom i sa sâmim sobom (pri čemu je on sâm upravo 1). Od studenata se može čak čuti i veoma pogrešna definicija po kojoj su prosti brojevi "oni i samo oni prirodni brojevi koji su djeljivi s jedinicom i sa sâmim sobom" (u ovoj definiciji nedostaje jedna riječ "samo"), s obzirom da bi, prema toj definiciji, svaki prirodan broj bio prost (svaki je djeljiv s jedinicom i sa sâmim sobom). Ovo možda djeluje kao nepotreban komentar, ali zaista se mnogo studenata čuje da ponude ovakvu "definiciju".

Kada smo raščistili s tim kada je broj prost, prvo ćemo napisati traženu funkciju "`DaLiJeProst`" koja ispituje da li je broj n zadan kao parametar prost ili ne. Iz definicije je očigledno da nijedan cijeli broj manji od 2 (0, 1 i svi negativni brojevi) nije prost, tako da iz funkcije odmah možemo vratiti logičku vrijednost "netačno" ukoliko je $n < 2$. Nakon toga, prva ideja koja se nameće je da petljom koja ide od 1 do n testiramo da li je n djeljiv s brojačem petlje, i ukoliko jeste, da uvećavamo neki brojač (koji ćemo na početku postaviti na nulu). Ukoliko na kraju brojač ima vrijednost 2, broj je prost, inače nije. Međutim, ovo je vrlo loša ideja, jer se petlja uvijek izvodi n puta, što je sporo ako je n veliko. Znatno je bolja ideja ići petljom od 2 do $n-1$ (1 i n su svakako djelioci od n), te prekinuti petlju uz vraćanje rezultata "netačno" čim ustanovimo da je n djeljiv s brojačem petlje. Ukoliko se petlja "izvrti" do kraja, bez prekida, djelilaca nema (osim 1 i n) i možemo vratiti logičku vrijednost tačno. Dobra strana ovog rješenja je što će petlja biti prekinuta čim se nađe prvi djelilac (recimo, za $n = 2115436255$, koji je očito djeljiv s 5, petlja će biti prekinuta već nakon 4 prolaza, umjesto nakon 2115436253 prolaza, što je ubrzanje od oko 528859063 puta)! Nažalost, ukoliko je n prost, petlja će se i dalje morati "izvrtiti" $n-2$ puta, što može biti presporo ako je n velik. Međutim, ogromno ubrzanje se može dobiti ukoliko primijetimo da petlja ne mora ići sve do $n-1$, pa čak ni do $n/2$ (što je optimizacija koju mnogi studenti primijete), nego je dovoljno da ide samo do $\sqrt{n}$. Zaista, ako je d djelilac od n, tada je n/d također djelilac od n (jer je $n/(n/d) = d$). Pri tome, ukoliko je $d \geq \sqrt{n}$, tada je $n/d \leq \sqrt{n}$. Dakle, ukoliko broj n uopće ima djelilaca (osim 1 i samog sebe), barem jedan od njih mora biti manji od $\sqrt{n}$. Odnosno, ukoliko ne nađemo djelilac u opsegu od 2 do $\sqrt{n}$, tada djelilaca uopće nema. Koliko ubrzanje se ovim postiže, možemo vidjeti recimo na primjeru broja $n = 2038074743$. Bez opisane optimizacije, petlja bi se morala "izvrtiti" 2038074741 puta da se ustanovi da je on prost, a s ovom optimizacijom dovoljna su 45144 prolaza, što je ubrzanje od oko 45145 puta. Slijedi prva verzija funkcije "`DaLiJeProst`" vrijedna pažnje (uskoro ćemo napisati i neznatno bolju verziju):

```
bool DaLiJeProst(int n) {
  if(n < 2) return false;
  int korijen = int(std::sqrt(n));
  for(int i = 2; i <= korijen; i++)
    if(n % i == 0) return false;
  return true;
}
```

U ovoj izvedbi treba ukazati na neke detalje. Prvo, računanje korijena je izvedeno izvan petlje, umjesto pisanja uvjeta poput "`i <= std::sqrt(n)`" kao uvjeta za kontrolu petlje. Naime, ukoliko bismo takav uvjet napisali kao uvjet za kontrolu petlje, korijen bi se iznova računao prilikom svakog prolaska kroz petlju, odnosno onoliko puta koliko se puta petlja izvede, što bi bilo vrlo nepotrebno usporavanje. Doduše, dobri kompajleri, uz uključenu optimizaciju (koja se opcionalno uključuje ili isključuje negdje u postavkama kompajlera), mogli bi zaključiti da se stalno računa jedna te ista vrijednosti i izbjeći ponavljanje računanja. Međutim, bolje je ne osloniti se na to da će kompajler to učiniti, kada to lako možemo izvesti sami. Drugo, primijetimo da je za čuvanje korijena iskorištena cjelobrojna, a ne realna promjenljiva "`korijen`". To je urađeno zbog toga što je poređenje dvije cjelobrojne vrijednosti daleko efikasnije nego poređenje jedne cjelobrojne i jedne realne (ili dvije realne) vrijednosti, a decimalni dio korijena nam ovdje zaista ne treba. Također, treba primijetiti da smo rezultat funkcije "`sqrt`" eksplicitnom pretvorbom (konverzijom) pretvorili u tip "`int`" (što dovodi do odbacivanja decimala). Strogo uzevši, ova eksplicitna pretvorba nije bila neophodna, jer bi svakako decimale bile odsječene pri pokušaju smještanja necjelobrojne vrijednosti u cjelobrojnu promjenljivu (automatska pretvorba). Međutim, neki kompajleri bi prijavili upozorenje (mada ne i grešku) da je dodjela necjelobrojne vrijednosti cjelobrojnoj promjenljivoj "sumnjiva" i da će to dovesti do gubitka informacija u slučaju da programer toga nije svjestan. Ovako smo eksplicitnom

konverzijom "ušutkali" kompajler, jer smo jasno rekli da nam je upravo to i namjera. Konačno, veoma je važno napomenuti da bi dodavanje "`else`" ispred posljednje naredbe "`return true;`" bila *katastrofalna greška*. Naime, bitno je uočiti da se ova naredba zapravo nalazi izvan petlje, odnosno izvršava se tek kada se petlja završi (naravno, to vrijedi samo ukoliko nismo prijevremeno izašli ne samo iz nje, nego i iz čitave funkcije prethodnom naredbom "`return false;`", koja se izvršava samo pod uvjetom "`n % i == 0`". Dakle, važno je shvatiti da se unutar tijela petlje nalazi samo složena naredba "`if(n % i == 0) return false;`", dok je naredba "`return true;`" izvan petlje. Ovo bi se jasnije vidjelo da je ta složena naredba ubačena u blok (tj. napisana unutar vitičastih zagrada), ali blokovi nisu neophodni kada se tijelo sastoji od samo jedne naredbe. Šta bi se desilo dodavanjem "`else`" ispred "`return true;`"? Kako "`else`" nikada ne može postojati kao samostalna naredba, nego samo kao "amandman" (dodatak) na "`if`" naredbu koja joj neposredno prethodi, to bi konstrukcija "`else return true;`" bilo shvaćena kao dodatak na prethodnu "`if`" naredbu, odnosno cjelokupna konstrukcija "`if(n % i == 0) return false; else return true;`" bila bi shvaćena kao jedna velika složena naredba. Kao takva, ona bi čitava pripadala tijelu petlje (jer ne može pola naredbe biti unutar petlje, a pola izvan nje). Drugim riječima, bilo bi isto kao da je kompletna ta konstrukcija stavljena unutar bloka (koji ponovo nije neophodan, jer se radi o samo jednoj naredbi, doduše uz "prikačeni amandman"). Dakle, u prvom prolazu kroz petlju, biće testirano da li je "`n`" djeljiv s 2. Ukoliko jeste, funkcija će biti prekinuta uz vraćanje "netačno". Međutim, u suprotnom, odmah skačemo u "`else`" dodatak, i prekidamo funkciju uz vraćanje "tačno". Drugim riječima, petlja uopće neće dobiti priliku da se izvrši više od jedanput, a tako napisana funkcija vratiće "tačno" za svaki neparan prirodan broj!

Kao rezime, možemo reći da konstrukcijama oblika "`if(...) return ...; else return ...;`" jednostavno nije mjesto unutar petlji (one čine da petlja neće dobiti priliku da se izvrši više od jedanput). Ova greška je toliko česta kod početnika da prosto traži *psihološko istraživanje zbog čega je tako*. Jedan mogući razlog je što mnogi često nisu svjesni da je u prethodno napisanoj funkciji konstrukcija "`return true;`" izvan petlje. Nažalost, to rade i mnogi koji toga budu svjesni, tako da to nije jedini razlog (potpuno razumijevanje ovog fenomena ostaje i dalje misterija za proučavanje). Zanimljivo je da se opasnost od nehotičnog stavljanja "`else`" ne bi mogla dogoditi da smo kojim slučajem konstrukciju "`if(n % i == 0) return false;`" ubacili u blok, jer bi tada stavljanje "`else`" ispred "`return true;`" (koji bi ostao izvan bloka) izazvao sintaksnu grešku. Naime, blokovi imaju lokalnu vidljivost (stvari unutar bloka ne vide se izvan njega), tako da "`else`" ne bi "vidio" odgovarajuću "`if`" naredbu s kojom bi se povezao. Početnicima se stoga savjetuje da upravo tako i rade. Međutim, jako mnogo programa je napisano bez korištenja blokova kad blok sadrži samo jednu naredbu, i mnogi programeri tako i rade, tako da se treba odmah navikavati kako korektno tumačiti programe u kojima nisu korišteni blokovi tamo gdje to nije neophodno.

Nakon ove malo duže elaboracije o vrlo čestoj mogućoj greški, razmotrimo da li se napisana funkcija "`DaLiJeProst`" može poboljšati. Na prvom mjestu, zanimljivo je da je funkciju "`sqrt`" moguće izbjeći praktično bez gubitka na efikasnosti i na taj način izbjeći u potpunosti potrebu za bibliotekom s zaglavljem "`cmath`", kao i aritmetiku s realnim brojevima. Naime, uvjet oblika $i \leq \sqrt{n}$ uz nenegativne n i i potpuno je matematički ekvivalentan uvjetu $i^2 \leq n$, u kojem se korjenovanje ne javlja. Ovo navodi na ideju da se uvjet "`i <= korijen`" u funkciji zamijeni uvjetom "`i * i <= n`". Nažalost, to *nije dobra ideja*. Naime, u jezicima C i C++ kod svih aritmetičkih operatora vrijedi konvencija da ukoliko su oba operanda tipa "`int`", rezultat je također tipa "`int`". Inače, kad god su operandi istog tipa, rezultat je u većini slučajeva tog istog tipa, mada ima izuzetaka (recimo, ukoliko su oba operanda tipa "`char`", rezultat nije tipa "`char`", nego "`int`"). Stoga se može desiti da, ukoliko je vrijednost promjenljive "`i`" isuviše velika, rezultat množenja "`i * i`" bude prevelik da može stati u tip "`int`". To će izazvati *prekoračenje opsega* i *nedefinirano ponašanje*. Ova opasnost je vrlo realna, jer će se to desiti kadgod je $i > \sqrt{M}$, gdje je M najveći broj koji može stati u tip "`int`". Kod većine današnjih kompajlera je $M = 2^{31} - 1 = 2147483647$, tako da bi problem nastao već za $i \geq 46341$, što uopće nije mnogo. Srećom, u ovom slučaju problem se vrlo lako rješava, jer je uvjet $i^2 \leq n$ za pozitivno i matematički potpuno ekvivalentan uvjetu $i \leq n/i$ koji nikada ne može izazvati prekoračenje (s obzirom da je za pozitivne cjelobrojne n i i, n/i manji i od n i od i). To daje sljedeću verziju funkcije "`DaLiJeProst`":

```
bool DaLiJeProst(int n) {
  if(n < 2) return false;
  for(int i = 2; i <= n / i; i++)
    if(n % i == 0) return false;
  return true;
}
```

Neko bi mogao (ne sasvim bez osnova) prigovoriti da je ova verzija funkcije "`DaLiJeProst`" sporija od prethodne verzije, s obzirom da se u uvjetu petlje stalno računa količnik "`n / i`" (koji se ne može računati izvan petlje, jer se "`i`" mijenja unutar petlje). Međutim, cjelobrojno dijeljenje je toliko brza operacija na današnjim računarskim arhitekturama da se degradacija u perfomansama ne samo da ne može primijetiti, nego čak ne može ni izmjeriti.

Može li bolje? Lako je ostvariti dodatno ubrzanje s faktorom 2. Naime, svi parni brojevi osim 2 sigurno nisu prosti, tako da ako je n paran broj različit od 2, možemo odmah reći da broj nije prost (sâm broj 2 je, naravno, prost). U suprotnom, možemo petljom tragati za djeliocima polazeći od 3, ali u koracima po 2 (tj. preskačući sve parne brojeve), jer ako broj nije paran, sigurno mu nijedan parni broj nije djelilac. Time se broj neophodnih prolazaka kroz petlju prepolovljava. Na taj način dolazimo do sljedeće, vremenski optimizirane verzije:

```
bool DaLiJeProst(int n) {
  if(n < 2 || n != 2 && n % 2 == 0) return false;
  for(int i = 3; i <= n / i; i += 2)
    if(n % i == 0) return false;
  return true;
}
```

Treba obratiti pažnju da se složeni uvjet "`n < 2 || n != 2 && n % 2 == 0`" zbog prioriteta operatora zapravo interpretira kao "`(n < 2) || ((n != 2) && (n % 2 == 0))`" i treba ga čitati kao "ako je n manji od 2, ili ako je n različit od 2 i paran". Šta ukoliko je $n = 2$? Ovaj uvjet tada nije tačan i dolazimo do petlje. Međutim, petlja se neće izvršiti nijedanput, jer uvjet petlje neće biti tačan već za početnu vrijednost brojača petlje, osnosno čitava petlja će biti preskočena, tako da će funkcija korektno vratiti "tačno".

Moguće je ostvariti i ubrzanje za još oko 30 % ukoliko primijetimo da svi prosti brojevi osim 2 i 3 moraju imati oblik $6k-1$ ili $6k+1$ ze neki prirodan broj k, što omogućava da testiramo samo brojeve ovog oblika, koristeći dvije neovisne petlje u kojima bi se brojač petlje povećavao s korakom 6 (probajte ovo uraditi sami kao vježbu). Ovim smo došli do granice šta je moguće uraditi samo korištenjem elementarne matematike. Još brže testiranje prostosti je zaista izvodljivo, ali traži duboko poznavanje teorije brojeva, što većini studenata nije poznata materija.

Pređimo sada na pisanje funkcije "`ProstiBrojeviUOpsegu`", koja kao rezultat vraća vektor koji se sastoji od svih prostih brojeva u intervalu od "`a`" do "`b`" uključivo (tj. svih prostih brojeva koji su veći ili jednaki od "`a`", a manji ili jednaki od "`b`"). Problem djeluje vrlo jednostavan. Dovoljno je na početku kreirati prazan vektor, a zatim petljom testirati ih na prostost (pozivom ranije napisane funkcije "`DaLiJeProst`") i dodavati u vektor (pozivom funkcije "`push_back`") ukoliko se ustanovi da jesu prosti. To bi bilo sve da nema zahtjeva da sve mora raditi ispravno za sve ulazne podatke koji mogu stati u tip "**`int`**". Problem nastaje ukoliko je gornja granica petlje "`b`" tačno jednaka najvećoj vrijednosti koja može stati u tip "**`int`**". Kad brojač petlje dođe tačno do te vrijednosti, njegovo dalje povećanje za 1 izazove prekoračenje. Prekoračenje se u aritmetici s predznakom smatra *nedefiniranim ponašanjem*, tako da se *ne zna šta će se dogoditi nakon tog prekoračenja*. Šta god da se dogodi, ne smijemo se osloniti na to da će se baš to dogoditi (pročitajte dodatak o prekoračenjima da se upoznate sa nekim zaista nevjerovatnim iznenađenjima koja se mogu pojaviti). Na većini kompajlera će takav pokušaj povećanja rezultirati najmanjim (tj. negativnim a najvećim po modulu) brojem koji može stati u "**`int`**" i petlja će nastaviti dalje, umjesto da se zaustavi. Ovaj problem se rješava tako što ćemo na kraj petlje dodati uvjet da se petlja prekida (pomoću naredbe "**`break`**") čim njen brojač dostigne gornju granicu "`b`", prije nego što se izvrši povećanje brojača (i eventualno uzrokuje prekoračenje). Uz taj dodatak, izvedba funkcije "`ProstiBrojeviUOpsegu`" mogla bi izgledati ovako:

```
std::vector<int> ProstiBrojeviUOpsegu(int a, int b) {
  std::vector<int> v;
  for(int i = a; i <= b; i++) {
    if(DaLiJeProst(i)) v.push_back(i);
    if(i == b) break;                        // Ovo je zbog INT_MAX...
  }
  return v;
}
```

Ostaje još da se napiše glavni program. Nakon unosa ulaznih podataka, pozivamo napisanu funkciju "`ProstiBrojeviUOpsegu`" i njenim rezultatom inicijalizoramo neku vektorsku promjenljivu koja će čuvati taj rezultat. Ta promjenljiva bi trebala biti tipa "`std::vector<int>`" (naravno, za tu svrhu je

potrebno uključiti biblioteku sa zaglavljem "`vector`" u program) ali kraće je da koristimo "**`auto`**" deklaraciju, što možemo koristiti kadgod želimo kreirati inicijaliziranu promjenljivu nekog tipa nekim izrazom koji je upravo tog tipa (npr. rezultatom funkcije koja vraća rezultat baš tog tipa). Takve deklaracije mogu uštediti pisanje, ali s njima ne treba pretjerivati (pogotovo ne treba koristiti "**`auto`**" deklaracije samo s ciljem da "zadovoljimo" kompajler, a da pri tome ni sami nismo sigurni kojeg će tipa ta promjenljiva zaista biti). Ukoliko je dobijeni vektor prazan (što možemo saznati poređenjem rezultata funkcije "`size`" pozvane nad tim vektorom s nulom), ispisujemo da nema prostih brojeva u zadanom rasponu. U suprotnom, ispisujemo sadržaj vektora pomoću petlje, element po element (elementima pristupamo preko indeksa). Treba samo voditi računa da iza svih elemenata treba ispisati zarez, osim iza posljednjeg (kada treba ispisati novi red). Da li je element posljedni ili ne, možemo saznati poređenjem indeka petlje s indeksom posljednjeg elementa, koji je za 1 manji od veličine vektora. Pri tome, naredbu "**`if`**" možemo izbjeći korištenjem ternarnog "`?:`" operatora (podsjetimo se da izraz oblika "x `?` y `:` z" ima vrijednost "y" ukoliko je "x" tačan, odnosno vrijednost "z" ukoliko je "x" netačan). Treba samo napomenuti da ukoliko koristimo ovu ideju, tada prelaz u novi red moramo izvesti slanjem sekvence "`\n`" a ne pomoću manipulatora "`endl`", s obzirom da operator "`?:`" ne prihvata manipulator "`endl`" kao operand. Ovo je sve izvedeno u sljedećem programu, u kojem smo kao komentare upisali mjesta na koje bi trebalo ubaciti ranije napisane funkcije (koje ovdje nismo prepisivali, da izbjegnemo ponavljanje):

```
#include <iostream>
#include <vector>

// Ubaciti ovdje definiciju funkcije "DaLiJeProst”

// Ubaciti ovdje definiciju funkcije "ProstiBrojeviUOpsegu"

int main() {
  int p, q;
  std::cout << "Unesite pocetnu i krajnju vrijednost: ";
  std::cin >> p >> q;
  auto v = ProstiBrojeviUOpsegu(p, q);
  if(v.size() == 0)
    std::cout << "Nema prostih brojeva u rasponu od " << p << " do " << q << "!\n";
  else {
    std::cout << "Prosti brojevi u rasponu od " << p << " do " << q << " su: ";
    for(int i = 0; i < v.size(); i++)
      std::cout << v[i] << (i != v.size() - 1 ? ", " : "\n");
  }
  return 0;
}
```

5. Napisati dvije funkcije "`ProstiFaktori`" i "`ProstiFaktoriSPonavljanjem`", vezane za nalaženje svih prostih faktora nekog broja. Obje funkcije kao jedini parametar primaju neki cijeli broj n. Prva funkcija vraća vektor koji se sastoji od svih prostih faktora broja n, sortiran po veličini, dok druga funkcija radi slično, samo što se u vraćenom vektoru svaki prosti faktor ponavlja onoliko puta koliko puta se on javlja u datom broju. Recimo, ukoliko se funkciji zada broj $n = 77265552$, prva funkcija treba da vrati vektor čiji su elementi 2, 3, 7, 13 i 19, dok druga funkcija treba da vrati vektor čiji su elementi 2, 2, 2, 2, 3, 7, 7, 7, 13, 19 i 19, s obzirom da rastava ovog broja na proste faktore glasi $n = 2^3 \cdot 3 \cdot 7^2 \cdot 13 \cdot 19^2$. Napisane funkcije iskoristite u testnom programu koji za broj unesen s tastature ispisuje njegove proste faktore međusobno razdvojene razmakom, u skladu s primjerom dijaloga prikazanog ispod (ukoliko broj nema prostih faktora, ispisuje se komentar "Broj nema prostih faktora!", što se može desiti samo ako je broj 1, ili 0, ili negativan):

```
Unesite broj: 77265552
Prosti faktori: 2 3 7 13 19
Prosti faktori s ponavljanjem: 2 2 2 2 3 7 7 7 13 19 19
```

```
Unesite broj: 1
Broj nema prostih faktora!
```

Rješenje:

Jedno vrlo loše rješenje za funkciju "`ProstiFaktori`" je da petljom testiramo sve brojeve u opsegu od 2 do n i ukoliko ustanovimo da je n djeljiv s tim brojem i da je taj broj ujedno prost, dodajemo taj broj u inicijalno prazan vektor. Testiranje prostosti možemo vršiti pomoću funkcije "`DaLiJeProst`" iz prethodnog zadatka. To daje sljedeću implementaciju:

```
std::vector<int> ProstiFaktori(int n) {
  std::vector<int> v;
  for(int i = 2; i <= n; i++)
    if(n % i == 0 && DaLiJeProst(i)) v.push_back(i);
  return v;
}
```

Ova izvedba je loša, jer je neefikasna. Naime, mnogo vremena se troši na testiranje prostosti svakog od eventualnih djelioca broja n. Mada izgleda da je to neophodno, vidjećemo uskoro da postoji način da lako nalazimo proste djelioce bez potrebe da ih zaista eksplicitno testiramo na prostost (ovo podsjeća na one zagonetke tipa "reci da si student ETF-a bez da eksplicitno kažeš da si student ETF-a). Međutim, prije toga, treba ukazati na jedan detalj u ovako napisanoj funkciji. Naime, ukoliko bismo uvjet "`n % i == 0 && DaLiJeProst(i)`" zamijenili matematički ekvivalentnim uvjetom "`DaLiJeProst(i) && n % i == 0`" (ekvivalencija slijedi iz činjenice da je, barem s aspekta matematike, konjunkcija komutativna operacija), funkcija bi bila neuporedivo sporija. Recimo, za primjer iz postavke zadatka $n = 77265552$, ona bi bila sporija više od 330 puta! Razlog za to je sljedeći. U uvjetu "`n % i == 0 && DaLiJeProst(i)`" prvo se testira da li je "`i`" djelilac od "`n`". Ukoliko nije, čitav uvjet se proglašava netačnim, tako da se "`i`" uopće neće ni testirati na prostost (ovakav način izračunavanja naziva se *kratkospojno izračunavanje*, ili engl. *short circuit evaluation*). Tek ukoliko "`i`" jeste djelilac od "`n`", "`i`" se testira na prostost i od rezultata tog testiranja zavisiće da li je čitav uvjet tačan ili ne. S druge strane, u uvjetu "`DaLiJeProst(i) && n % i == 0`" prvo se testira da li je "`i`" prost broj. Ukoliko nije, čitav uvjet se proglašava netačnim. Međutim, ukoliko jeste, testira se da li je "`i`" djelilac od "`n`" i od rezultata tog testiranja zavisiće da li je čitav uvjet tačan ili ne. Kao rezime, u prvom uvjetu svaka vrijednost "`i`" se testira da li je djelilac od "`n`", pri čemu se test na prostost obavlja jedino ukoliko je taj test bio uspješan. S druge strane, u drugom uvjetu, svaka vrijednost "`i`" se testira na prostost, pri čemu se test na djeljivost obavlja jedino ukoliko je taj test bio uspješan. Razlika nastaje zbog toga što je test na prostost (koji se u drugom uvjetu obavlja za svaku vrijednost "`i`") neuporedivo složeniji (i stoga sporiji) od običnog testa na djeljivost. To objašnjava ogromnu razliku u efikasnosti. Ovaj primjer ilustrira koliko ponekad naizgled minorna promjena u redoslijedu izvođenja operacija može utjecati na efikasnost.

Sada ćemo pokazati kako je moguće napraviti mnogo bolje rješenje, u kojem se uopće ne koristi test na prostost. Za to, potrebno je prvo primijetiti sljedeću činjenicu. Neka je d najmanji djelilac broja n. Tada je d sigurno prost broj. Zaista, kada d ne bi bio prost broj, on bi morao imati neki prost faktor p manji od d. Broj n bi tada bio također djeljiv i sa p (ako je jedan broj djeljiv drugim brojem, onda je on djeljiv i sa svakim djeliocem tog drugog broja), što protivrječi pretpostavci da je d najmanji djelilac broja n (jer bi tada p bio manji djelilac nego d). Dakle, d mora biti prost broj.

Na osnovu provedenog razmatranja, algoritam možemo zasnovati ovako. Prvo tražimo najmanji djelilac od n, tako što ćemo, polazeći od $d = 2$, u while petlji povećavati d sve dok n ne postane djeljiv s d. Kao što smo ranije vidjeli, d je tada prosti djelilac od n, tako da ga ubacujemo u vektor. Nakon toga, u novoj while petlji uzastopno dijelimo n s d, sve dok je to dijeljenje moguće. Na taj način "čistimo" broj n od svih pojava faktora d u njemu, tako da će novodobijeni broj n imati sve iste faktore kao i ranije, osim faktora d. Čitav postupak ponavljamo s novim brojem n sve dok je n veći od 1, kada se postupak završava (za tu svrhu, sve što smo ranije rekli treba biti unutar još jedne while petlje koja se izvršava sve dok je n veći od 1). To daje sljedeću implementaciju:

```
std::vector<int> ProstiFaktori(int n) {
  std::vector<int> v;
  int d = 2;
  while(n > 1) {
    while(n % d != 0) d++;
    v.push_back(d);
    while(n % d == 0) n /= d;
  }
  return v;
}
```

Što se tiče funkcije "`ProstiFaktoriSPonavljanjem`", ona se može izvesti trivijalnom modifikacijom prethodne funkcije. Naime, umjesto uzastopnog dijeljenja n s d u while petlji, potrebno je obaviti samo jedno dijeljenje. Ukoliko se faktor d javlja više od jedanput, on će biti ponovo nađen, sve dok se ne iscrpe sva njegova pojavljivanja. To vodi ka sljedećoj izvedbi:

```
std::vector<int> ProstiFaktoriSPonavljanjem(int n) {
  std::vector<int> v;
  int d = 2;
  while(n > 1) {
    while(n % d != 0) d++;
    v.push_back(d);
    n /= d;
  }
  return v;
}
```

Ostaje još da se napiše glavni program

```
#include <iostream>
#include <vector>

// Ubaciti ovdje definiciju funkcije "ProstiFaktori"

// Ubaciti ovdje definiciju funkcije "ProstiFaktoriSPonavljanjem"

int main() {
  int n;
  std::cout << "Unesite broj: ";
  std::cin >> n;
  auto v = ProstiFaktori(n);
  if(v.size() == 0)
    std::cout << "Broj nema prostih faktora!\n";
  else {
    std::cout << "Prosti faktori: ";
    for(int d : v) std::cout << d << " ";
    std::cout << "\nProsti faktori s ponavljanjem: ";
    for(int d : ProstiFaktoriSPonavljanjem(n)) std::cout << d << " ";
  }
}
```

Treba primijetiti da je za ispis sadržaja vektora iskorištena rasponska for petlja, što je ovdje moguće uraditi, jer se iza svakog elementa ispisuje isti separator (razmak). Također, treba primijetiti da je rezultat funkcije "`ProstiFaktori`" sačuvan u promjenljivoj "`v`", što je mudro s obzirom da se ona koristi na dva mjesta kasnije, dok rezultat funkcije "`ProstiFaktoriSPonavljanjem`" uopće ne čuvamo ni u kakvoj promjenljivoj. Umjesto toga, rasponsku for petlju izvodimo direktno nad bezimenim vektorom koji se dobija kao rezultat poziva ove funkcije. Generalno, treba izbjegavati upotrebu promjeljivih za smještanje bilo kakvih vrijednosti koje će biti odmah upotrijebljene na mjestu na kojem su i nastale, a koje neće biti upotrijebljene ni na jednom drugom mjestu.

6. Napisati dvije funkcije "`TestPerioda`" i "`OdrediOsnovniPeriod`", vezane za testiranje periodičnosti slijeda brojeva pohranjenih u vektoru i nalaženje osnovnog perioda tog slijeda (u slučaju da slijed jeste periodičan). Prva funkcija prima dva parametra, pri čemu je prvi parametar vektor realnih brojeva, dok je drugi parametar cijeli broj p. Funkcija treba da vrati kao rezultat logičku vrijednost "tačno" ili "netačno" ovisno od toga da li slijed brojeva pohranjen u vektoru periodičan sa periodom p ili ne. Druga funkcija prima samo jedan parametar koji je vektor realnih brojeva, a ona kao rezultat vraća osnovni period slijeda pohranjenog u vektoru ukoliko je slijed periodičan, ili nulu ukoliko slijed nije periodičan. Napisane funkcije treba iskoristiti u glavnom programu u kojem se unose elemente s tastature u neki vektor sve dok se sa tastature ne unese nula, koja označava kraj unosa (ta nula se ne smješta u vektor). Nakon završetka unosa, program treba pozvati funkciju "`OdrediOsnovniPeriod`" sa ciljem da utvrdi da li se elementi periodično ponavljaju ili ne, nakon čega ispisuje odgovarajuči komentar na ekranu (informaciju o dužini perioda, ili da elementi ne čine periodičan slijed). Pri tome, nije unaprijed poznato koliko će korisnik unijeti elemenata prije nego što unese nulu kao oznaku završetka unosa. Dijalozi između korisnika i programa trebali bi izgledati poput sljedećih (ovisno da li je slijed periodičan ili ne):

```
Unesite slijed brojeva (0 za kraj): 1 3 1 4 2 1 3 1 4 2 1 3 0
Slijed je periodican sa osnovnim periodom 5.
```

```
Unesite slijed brojeva (0 za kraj): 4 5 1 7 1 5 0
Slijed nije periodican!
```

Rješenje:

Da bismo riješili ovaj zadatak, prvo je potrebno jasno i precizno utvrditi šta znači da je neki slijed brojeva periodičan. Smatraćemo da slijed brojeva $a_1, a_2, \ldots, a_n$ ima period p ukoliko je $1 \leq p < n$ i ukoliko vrijedi da je $a_i = a_{i+p}$ za sve vrijednosti i za koje ova jednakost ima smisla (tj. za koju su oba indeksa i i $i+p$ legalni). Na primjer, slijed brojeva 1, 3, 1, 4, 2, 1, 3, 1, 4, 2, 1, 3 ima period 5, jer je $a_i = a_{i+5}$ za sve vrijednosti i takve da su oba indeksa i i $i+5$ unutar dozvoljenog opsega (tj. za i od 1 do 7 uključivo). Isti slijed također ima i period 10. Slijed brojeva je periodičan ukoliko postoji makar jedan broj p koji je period tog slijeda, pri čemu je najmanji takav broj p osnovni period slijeda. Ukoliko takav broj p ne postoji, slijed nije periodičan. Na primjer, gore navedeni slijed brojeva je periodičan s osnovnim periodom 5, dok slijed brojeva 4, 5, 1, 7, 1, 5 nije periodičan.

Sada kada je jasno šta je period slijeda, lako je napraviti funkciju koja testira da li slijed pohranjen u nekom vektoru ima zadani period p. Prvo, jasno je da je odgovor odrečan ako je p manji od 1, ili veći od broja elemenata u slijedu. Ukoliko to nije slučaj, možemo petljom proći kroz sve indekse i za koje su i indeksi i i $i+p$ legalni (odnosno u opsegu od 0 do $n-1$, gdje je n broj elemenata u vektoru). Da bi i i $i+p$ bili u opsegu od 0 do $n-1$, i treba biti u opsegu od 0 do $n-p-1$. Ukoliko za makar jedan takav indeks i vrijedi da je $a_i \neq a_{i+p}$, slijed nije periodičan s periodom p, tako da možemo prekinuti funkciju i vratiti "netačno". Međutim, ukoliko petlja uspije doći do kraja (a da prije toga nije nasilno prekinuta izlaskom iz funkcije), to znači da nismo našli nijedan indeks i u razmatranom opsegu takav da vrijedi $a_i \neq a_{i+p}$, odnosno za sve indekse i u razmatranom opsegu vrijedi $a_i = a_{i+p}$. Prema definiciji, p je tada period slijeda. To daje sljedeću izvedbu funkcije "`TestPerioda`" (treba primijetiti da bismo i u ovoj izvedbi mogli imati isti problem o kojem smo govorili u prethodnom zadatku ukoliko bismo ubacili suvišni "**`else`**" ispred "**`return true`**"):

```
bool TestPerioda(std::vector<double> v, int p) {
  if(p < 1 || p >= v.size()) return false;
  for(int i = 0; i < v.size() - p; i++)
    if(v[i] != v[i + p]) return false;
  return true;
}
```

Razmotrimo sada kako naći osnovni period slijeda, ukoliko takav postoji. Na prvi pogled, problem djeluje težak. Potrebno je naći najmanji broj p koji zadovoljava uvjet da je period. Pri tome, nije dovoljno naći najmanji p takav da za neki indeks i vrijedi $a_i = a_{i+p}$, nego ova jednakost mora vrijediti za sve indekse i takve da su i i i $i+p$ validni indeksi. Slijedi da nam je neophodna neka varijanta dvostruko ugniježdenih petlji, pri čemu ćemo u jednoj petlji tragati za p, a u drugoj testirati da li je traženi uvjet ispunjen za sve validne vrijednosti i (za šta opet treba petlja). Međutim, sve ovo može mnogo lakše ukoliko iskoristimo već napisanu funkciju "`TestPerioda`". Naime, dovoljno je u petlji testirati redom sve vrijednosti p od 1 do n pozivajući funkciju "`TestPerioda`" da utvrdimo da li je p period ili ne. Ukoliko jeste, prekidamo funkciju i vraćamo p kao rezultat. Može se postaviti pitanje kako smo sigurni da je to osnovni period. Razlog je što smo prije njega ispitali sve manje vrijednosti i nijedna od njih nije bila period (da jeste bila, funkcija bi bila prekinuta i ne bismo uopće došli do tog mjesta). Međutim, ukoliko petlja ispita sve vrijednosti p od 1 do n i nijedna od njih nije period (tako da ne dođe do prijevremenog prekida funkcije), slijed nije periodičan i možemo vratiti 0 kao rezultat. Sve ovo vodi ka sljedećoj vrlo jednostavnoj izvedbi (naravno, ubacivanje "**`else`**" ispred "**`return 0`**" bi i ovdje napravilo problem):

```
int OdrediOsnovniPeriod(std::vector<double> v) {
  for(int p = 1; p < v.size(); p++)
    if(TestPerioda(v, p)) return p;
  return 0;
}
```

Možemo primijetiti da ukoliko za makar jednu vrijednost i ne vrijedi $a_i = a_{i+p}$, slijed sigurno nema period p. Stoga možemo izvršiti minornu optimizaciju i na početku testirati ovaj uvjet za jednu fiksnu vrijednost indeksa, recimo za $i = 0$. Tek ukoliko je za nju ova jednakost ispunjena, ima smisla pozivati funkciju "`TestPerioda`" (u suprotnom znamo da će ona sigurno vratiti "netačno"). Stoga možemo izvesti minornu optimizaciju tako što ćemo unutar "**`if`**" naredbe prosti uvjet "`TestPerioda(v, p)`" zamijeniti složenim uvjetom "`v[0] == v[p] && TestPerioda(v, p)`". Naime, zbog načina kako radi operator konjunkcije "`&&`", ukoliko je prvi dio uvjeta (tj. poduvjet "`v[0] == v[p]`") netačan, cijeli uvjet se proglašava netačnim, bez da se drugi dio uvjeta (tj. poziv funckije "`TestPerioda`") uopće

ispituje. Na taj način izbjegava se poziv funkcije "`TestPerioda`" u situacijama kada odmah znamo da će poziv dati rezultat "netačno".

Napomenimo da ćemo u Predavanju 4, kada upoznamo reference, vidjeti da je parametre funkcija "`TestPerioda`" i "`OdrediOsnovniPeriod`" bolje deklarirati kao "`const std::vector<double> &v`" umjesto kao "`std::vector<double> v`". Ta izmjena neće se odraziti na samu funkcionalnost ovih funkcija, ali hoće na efikasnost.

Ostaje da se napiše glavni program. Što se tiče unosa, s obzirom da ne znamo koliko brojeva će se unijeti, njih ćemo unositi u do-while petlji koju izvodimo sve dok se ne unese nula. Svaki uneseni broj dodajemo na kraj inicijalno praznog vektora, osim ako je taj broj 0 (nulu ne ubacujemo). Ostaje još samo da pozovemo napisanu funkciju "`OdrediOsnovniPeriod`" i da ispišemo njen rezultat, odnosno informaciju da slijed nije periodičan (ako je rezultat 0). To daje sljedeći program

```
#include <iostream>
#include <vector>

// Ubaciti ovdje definiciju funkcije "TestPerioda"

// Ubaciti ovdje definiciju funkcije "OdrediOsnovniPeriod"

int main() {
  std::vector<double> v;
  std::cout << "Unesite slijed brojeva (0 za kraj): ";
  double x;
  do {
    std::cin >> x;
    if(x != 0) v.push_back(x);
  } while(x != 0);
  int p = OdrediOsnovniPeriod(v);
  if(p == 0) std::cout << "Slijed nije periodican!\n";
  else std::cout << "Slijed je periodican sa osnovnom periodom " << p << ".\n";
  return 0;
}
```

7. Napisati funkciju "`IzdvojiElemente`" sa dva parametra, od kojih je prvi vektor cijelih brojeva (tipa "`int`"), a drugi logička vrijednost "tačno" ili "netačno". Funkcija treba da kao rezultat vrati novi vektor koji se sastoji od onih elemenata čija je suma cifara parna odnosno neparna, ovisno od toga da li drugi parametar ima vrijednost "tačno" ili "netačno". Elementi u novokreiranom vektoru trebaju biti u istom međusobnom poretku u kakvom su bili u izvornom vektoru. Za realizaciju funkcije obavezno treba koristiti funkciju "`push_back`". Napisanu funkciju treba iskoristiti u testnom programu koji traži da se prvo unese prirodan broj n, a nakon toga elementi nekog vektora koji ima n cjelobrojnih elemenata. Program zatim treba da kreira dva nova vektora i da u prvi vektor prepiše sve brojeve iz početnog vektora s parnom sumom cifara, a u drugi vektor sve brojeve iz početnog vektora s neparnom sumom cifara. Konačno, program treba da u jednom redu ispiše brojeve sa parnom sumom cifara (ili tri crtice ako takvih brojeva nema), a u drugom redu brojeve sa neparnim brojem cifara (ili tri crtice). Mogući dijalozi između programa i korisnika trebaju izgledati ovako:

```
Koliko zelite unijeti elemenata: 12
Unesite elemente: 15 23 18 -26 0 142 -333 73 11111 -312 9 5555555
15 -26 0 73 -312
23 18 142 -333 11111 9 5555555
```

```
Koliko zelite unijeti elemenata: 5
Unesite elemente: 24 73 112 -17 0
24 73 112 -17 0
---
```

```
Koliko zelite unijeti elemenata: 4
Unesite elemente: 32 -23 111 45
---
32 -23 111 45
```

Za sve manipulacije sa vektorima potrebno je koristiti isključivo rasponsku for petlju, osim za unos elemenata (za unos elemenata pomoću rasponske for petlje neophodno je korištenje referenci, koje se obrađuju tek na Predavanju 4). Može se pretpostaviti da će svi ulazni podaci biti korektni.

Rješenje:

S obzirom da se razvrstavanje vrši po sumi cifara, korisno je posebno napisati pomoćnu funkciju (nazvanu recimo "`SumaCifara`") koja nalazi sumu cifara broja zadanog kao parametar. Logika za nalaženje sume cifara je jednostavna. Posljednja cifra broja uvijek se dobija kao ostatak pri njegovom dijeljenju s 10, dok cjelobrojno dijeljenje broja s 10 daje novi broj koji se sastoji od svih istih cifara osim posljednje. Dakle, u petlji je potrebno stalno izdvajati posljednju cifru (nalaženjem ostatka pri dijeljenju s 10) i nadodavati je na neku sumu inicijalno postavljenu na 0, a nakon toga dijeliti broj s 10 i ponavljati postupak sve dok se ne iscrpe sve cifre (tj. dok broj ne postane 0). Ovdje samo ima jedna mala "zvrčka". Ukoliko je broj negativan, dobija se i negativan ostatak pri dijeljenju s 10, odnosno umjesto izvdojene posljednje cifre, dobiće se negirana posljednja cifra. Samim tim, i suma cifara će ispasti negativna. Stoga, ukoliko se dobije negativna suma cifara, treba joj samo obrnuti znak (za tu svrhu, da izbjegnemo if naredbu, možemo unutar "`return`" naredbe iskoristiti ternarni operator "`?:`"). Kao petlju, najprirodnije bi bilo upotrijebiti while petlju, s obzirom da ne znamo koliko će se ona puta izvršiti. Međutim, izvedbu možemo neznatno skratiti ukoliko umjesto nje upotrijebimo for petlju s izostavljenim prvim parametrom. Naime, poznato je da je konstrukcija "`while(`*uvjet*`)`" ekvivalentna konstrukciji "`for(;` *uvjet*`;)`" u kojoj je u for naredbi izostavljen prvi parametar (inicijalizacija) i treći parametar (izmjena). Međutim, ovdje nećemo izostaviti i prvi i treći parametar, nego samo prvi, a treći parametar ćemo iskoristiti za dijeljenje broja čiju sumu cifara tražimo s 10, tako da će unutar tijela petlje ostati samo nadodavanje izvdojene cifre. To vodi ka sljedećoj implementaciji:

```
int SumaCifara(int n) {
  int suma = 0;
  for(; n != 0; n /= 10) suma += n % 10;
  return suma < 0 ? -suma : suma;
}
```

Sada je lako napisati i traženu funkciju "`IzdvojiElemente`". Rasponskom for petljom prolazimo kroz sve elemente vektora. Za svaki element, testiramo da li je njegova suma cifara parna ili neparna, te ovisno od toga ubacujemo ga ili ne ubacujemo u pomoćni vektor (koji je na početku prazan). Da li treba ubacivati parne ili neparne elemente, ovisi od dodatnog parametra funkcije logičkog tipa. To dovodi do implementacije, koja može izgledati poput sljedeće (slično kao u prethodnom zadatku, i ovdje vrijedi napomena da je, iz razloga vezanih za efikasnost parametar funkcije bolje deklarirati kao "`const std::vector<double> &v`" umjesto kao "`std::vector<double> v`"):

```
std::vector<int> IzdvojiElemente(std::vector<int> v, bool par_nepar) {
  std::vector<int> v1;
  for(int x : v)
    if(SumaCifara(x) % 2 == !par_nepar) v1.push_back(x);
  return v1;
}
```

U prethodnoj implementaciji izveden je sitni trik zasnovan na operatoru negacije "`!`" i automatskoj pretvorbi logičkih vrijednosti u cjelobrojne u slučaju njihovog korištenja u kontekstu u kojem se očekuje brojčana vrijednost (razmislite sami kako ovaj trik radi). Ovaj trik omogućio nam je da izbjegnemo rogobatne konstrukcije poput

```
if(par_nepar && SumaCifara(x) % 2 == 0) v1.push_back(x);
else if(!par_nepar && SumaCifara(x) % 2 == 1) v1.push_back(x);
```

ili, još gore, konstrukcije poput

```
if(par_nepar) {
  if(SumaCifara(x) % 2 == 0) v1.push_back(x);
}
else if(!par_nepar)
  if(SumaCifara(x) % 2 == 1) v1.push_back(x);
```

Ove konstrukcije ne samo da su rogobatne, nego su i neefikasne, zbog toga što se u njima funkcija "`SumaCifara`" ponekad poziva dva puta, tako da se dva puta računa jedna te ista vrijednost. To bi se moglo izbjeći čuvanjem rezultata funkcije u nekoj pomoćnoj promjenljivoj. Pored toga, druga od gore prikazanih konstrukcija (ona s ugniježdenim if-else naredbama) zahtieva da se zaista mora koristiti blok (tj. omeđenje vitičastim zagradama), bez obzira što je u njemu samo jedna naredba. Naime, ukoliko bismo izostavili vitičaste zagrade, konstrukcija "`else`" bila bi uparena s pogrešnom

"**if**" naredbom (njom najbližom, odnosno drugom po redu), umjesto s prvom "**if**" naredbom (upotreba bloka čini drugu "**if**" naredbu "nevidljivom" na mjestu gdje se nalazi "**else**"). Dakle, ukoliko se izostave vitičaste zagrade, ova konstrukcija radi neispravno, mada je sintaksno ispravna! Ilustrativno je razmotriti i kako bi se mogla napisati funkcija "IzdvojiElemente" bez pisanja pomoćne funkcije "SumaCifara" trebalo bi ugraditi negdje ispred mjesta gdje se ona pozivala unutar funkcije "IzdvojiElemente", pri čemu bi se nađena suma cifara čuvala u nekoj promjenljivoj. Tako napisana funkcija mogla bi izgledati recimo ovako (razmotrite sami zbog čega je kreirana nova promjeljiva "n" koja na početku čuva kopiju vrijednosti iz promjenljive "x", umjesto da se sav postupak koji se radi s promjenljivom "n" obavlja direktno nad promjenljivom "x"):

```
std::vector<int> IzdvojiElemente(std::vector<int> v, bool par_nepar) {
  std::vector<int> v1;
  for(int x : v) {
    int n = x, suma = 0;
    for(; n != 0; n /= 10) suma += n % 10;
    if(suma_cifara < 0) suma = -suma;
    if(suma % 2 == !par_nepar) v1.push_back(x);
  }
  return v1;
}
```

Ostaje još samo da se napiše glavni program, što je posve jednostavno, slično kao u prethodnom zadatku. Za ispis elemenata korištena je rasponska for petlja, kao što je i traženo u postavci:

```
#include <iostream>
#include <vector>

// Ubaciti ovdje definiciju funkcije "SumaCifara" (ako je korištena kao pomoćna)

// Ubaciti ovdje definiciju funkcije "IzdvojiElemente"

int main() {
  int n;
  std::cout << "Koliko zelite unijeti elemenata: ";
  std::cin >> n;
  std::vector<int> a(n);
  std::cout << "Unesite elemente: ";
  for(int i = 0; i < n; i++) std::cin >> a[i];
  auto b = IzdvojiElemente(a, true), c = IzdvojiElemente(a, false);
  if(b.size() == 0) std::cout << "---\n";
  for(int x : b) std::cout << x << " ";
  std::cout << std::endl;
  if(c.size() == 0) std::cout << "---\n";
  for(int x : c) std::cout << x << " ";
  std::cout << std::endl;
  return 0;
}
```

8. Prepraviti prethodni program tako da se brojevi ispisuju razdvojeni zarezom a ne razmakom, pri čemu iza posljednjeg broja u svakom redu ne treba da bude zarez. Recimo, za iste ulazne podatke kao u prvom primjeru dijaloga iz prethodnog zadatka, dijalog između korisnika i programa trebalo bi ovaj put izgledati ovako:

```
Koliko zelite unijeti elemenata: 12
Unesite elemente: 15 23 18 -26 0 142 -333 73 11111 -312 9 5555555
15,-26,0,73,-312
23,18,142,-333,11111,9,5555555
```

Rješenje:

Ovdje je očigledno potrebno izmijeniti samo glavni program (tačnije, dio u kojem se vrši ispis vektora), dok sve pomoćne funkcije ostaju potpuno iste. Pri tome, kako je ispis u istom formatu kao u Zadatku 4, iskoristićemo ideju iz tog zadatka. Međutim, kako se u programu ispisuju dva vektora, dupliranje koda možemo izbjeći tako što ćemo napisati pomoćnu funkciju (nazvanu recimo "IspisiVektor") koja ispisuje sve elemente vektora razdvojene zarezima, s prelaskom u novi red umjesto zareza nakon ispisa posljednjeg elementa. Da u zadatku nije eksplicitno postavljeno ograničenje da se za svu obradu podataka osim za unos trebaju koristiti rasponske for petlje, tu funkciju bi najbolje bilo izvesti na isti način kao u Zadatku 4, odnosno ta funkcija bi mogla izgledati

recimo ovako (iz istih razloga koje smo već napominjali, njen parametar bi bilo bolje deklarirati kao "`const std::vector<double> &v`" umjesto kao "`std::vector<double> v`"):

```
void IspisiVektor(std::vector<int> v) {
  for(int i = 0; i < v.size(); i++)
    std::cout << v[i] << (i != v.size() - 1 ? "," : "\n");
}
```

Istu stvar je malo teže riješiti s rasponskom for petljom, s obzirom da u takvim petljama nema brojača pomoću kojih bismo mogli testirati da li smo došli do kraja ili ne (da odlučimo da li ispisati zarez ili preći u novi red). Ipak, moguće je i rješenje s rasponskom for petljom, pod uvjetom da uvedemo promjenljivu koja će imati ulogu "brojača", koji ćemo ručno ažurirati u svakom prolasku kroz petlju. Takva izvedba bi mogla izgledati recimo ovako:

```
void IspisiVektor(std::vector<int> v) {
  int i = 0;
  for(int x : v) {
    std::cout << x << (i != v.size() - 1 ? "," : "\n");
    i++;
  }
}
```

Ovu izvedbu je moguće skratiti ukoliko uvećanje promjenljive "`i`" izvedemo kao propratni efekat izraza u kojem se vrši njeno testiranje. Također, ukoliko bismo tu promjenljivu inicijalizirali na 1 umjesto na 0, ne bi nam u testu trebalo oduzimanje jedinice od veličine vektora (vrijeme za prelazak u novi red bilo bi taman onda kada vrijednost "`i`" postane jednaka veličini vektora). Na taj način, dolazimo do sljedeće optimizirane izvedbe:

```
void IspisiVektor(std::vector<int> v) {
  int i = 1;
  for(int x : v) std::cout << x << (i++ != v.size() ? "," : "\n");
}
```

Sada bi kompletan program mogao izgledati ovako:

```
#include <iostream>
#include <vector>

// Ubaciti ovdje definiciju funkcije "SumaCifara" (ako je korištena kao pomoćna)

// Ubaciti ovdje definiciju funkcije "IzdvojiElemente"

// Ubaciti ovdje definiciju funkcije "IspisiVektor"

int main() {
  int n;
  std::cout << "Koliko zelite unijeti elemenata: ";
  std::cin >> n;
  std::vector<int> a(n);
  std::cout << "Unesite elemente: ";
  for(int i = 0; i < n; i++) std::cin >> a[i];
  auto b = IzdvojiElemente(a, true), c = IzdvojiElemente(a, false);
  if(b.size() == 0) std::cout << "---\n";
  IspisiVektor(b);
  if(c.size() == 0) std::cout << "---\n";
  IspisiVektor(c);
  return 0;
}
```

9. Izvršiti prepravku prethodnog programa tako da se umjesto tipa "`vector`" koristi tip "`deque`" (tj. da se koristi dek umjesto vektora) i provjerom pokazati da nakon te prepravke sve i dalje radi potpuno isto. Zatim zamijeniti poziv funkcije "`push_back`" unutar funkcije "`IzdvojiElemente`" (koja se morala koristiti za njenu realizaciju) s pozivom funkcije "`push_front`" i ispitajte na koji način ta izmjena utječe na rezultate izvršavanja programa. Konačno, izvršite izmjene u funkciji "`IzdvojiElemente`" tako da se i dalje koristi funkcija "`push_front`", ali da rezultat koji daje funkcija bude isti kakav je bio u prethodnom zadatku (tako da i ispis koji će program proizvesti bude posve identičan kao u prethodnom zadatku). Pri tome, u funkciji nije dozvoljeno kreiranje ikakvih pomoćnih kontejnera (nizova, vektora, dekova, itd.).

Rješenje:

Prva prepravka koja se traži u zadatku traži zamjenu svake pojave riječi "`vector`" s riječi "`deque`" (uključujući i to da treba uključiti biblioteku sa zaglavljem "`<deque>`" umjesto "`<vector>`", Kako dek podržava sve operacije koje podržava i vektor i to s istim rezultatima i istim vidljivim efektima, nikakva vidljiva promjena u rezultatima koje prikazuje program neće se primijetiti. Ipak, nije sve potpuno isto. Naime, pristup elementima deka je sporiji nego pristup elementima vektora (tipično 2 do 3 puta), tako da će program raditi sporije. Međutim, to se ne može primijetiti osim u slučaju da vektor odnosno dek sadrži zaista veoma mnogo elemenata (recimo milion ili više). S vektorima ili dekovima male veličine, obje izvedbe su toliko brze da nije moguće uočiti razliku.

Ukoliko pored zamjene svih vektora s dekovima zamijenimo i poziv funkcije "`push_back`" s pozivom funkcije "`push_front`", tada će se svaki novi broj koji dodajemo dodavati na početak deka, umjesto na njegov kraj. Posljedica te promjene je što će svi elementi deka koji se vrati kao rezultat iz funkcije "`IzdvojiElemente`" biti u obrnutom redoslijedu u odnosu kakvi su bili kada je korištena funkcija "`push_back`". Samim tim, vidljiva posljedica će biti da će i elementi koji se ispisuju na ekran biti u obrnutom redoslijedu u odnosu na prethodni zadatak (odnosno, biće u obrnutom redoslijedu od redoslijeda unošenja). Tako će, za iste ulazne podatke kao u prvom dijalogu iz prethodnog zadatka, dijalog između korisnika i programa sada izgledati ovako:

```
Koliko zelite unijeti elemenata: 12
Unesite elemente: 15 23 18 -26 0 142 -333 73 11111 -312 9 5555555
-312,73,0,-26,15
5555555,9,11111,-333,142,18,23
```

Da bi program proizveo isti vidljivi efekat kao i program iz prethodnog zadatka, čak i uz zamjenu poziva funkcije "`push_back`" s pozivom funkcije "`push_front`", sâmo procesiranje elemenata ulaznog deka treba vršiti u obrnutom smjeru, tj. od posljednjeg elementa ka prvom. Nažalost, za tu svrhu nećemo moći koristiti rasponsku for petlju, jer ona uvijek ide u smjeru od prvog elementa ka posljednjem. Umjesto toga, koristićemo običnu for petlju. Slijedi kompletan prepravljeni program (funkciju "`SumaCifara`" ćemo preuzeti iz Zadatka 7, a radi kozistencije ćemo funkciji "`IspisiVektor`" promijeniti ime u "`IspisiDek`" i preimenovaćemo promjenljive koji se zovu "`v`" i "`v1`" u "`d`" i "`d1`"):

```
#include <iostream>
#include <deque>

// Ubaciti ovdje definiciju funkcije "SumaCifara" (ako je korištena kao pomoćna)

std::deque<int> IzdvojiElemente(std::deque<int> d, bool par_nepar) {
  std::deque<int> d1;
  for(int i = d.size() - 1; i >= 0; i--)
    if(SumaCifara(d[i]) % 2 == !par_nepar) d1.push_front(d[i]);
  return d1;
}

void IspisiDek(std::deque<int> d) {
  for(int i = 0; i < d.size(); i++) {
    std::cout << d[i];
    if(i != d.size() - 1) std::cout << ",";
    else std::cout << std::endl;
  }
}

int main() {
  int n;
  std::cout << "Koliko zelite unijeti elemenata: ";
  std::cin >> n;
  std::deque<int> a(n);
  std::cout << "Unesite elemente: ";
  for(int i = 0; i < n; i++) std::cin >> a[i];
  auto b = IzdvojiElemente(a, true), c = IzdvojiElemente(a, false);
  if(b.size() == 0) std::cout << "---\n";
  IspisiDek(b);
  if(c.size() == 0) std::cout << "---\n";
  IspisiDek(c);
  return 0;
}
```

10. Za dva prirodna broja m i n kažemo da čine prijateljski par brojeva ukoliko je suma svih djelilaca jednog broja (ne računajući njega samog) jednaka drugom broju i obrnuto. Na primjer, brojevi 220 i 284 su jedan takav par. Zaista, djelioci broja 220 (osim njega samog), su 1, 2, 4, 5, 10, 11, 20, 22, 44, 55 i 110, čija je suma 284, a isto tako suma svih djelioca broja 284 (osim njega samog) iznosi 220. Pored $(220, 284)$, još neki parovi prijateljskih brojeva su $(1184, 1210)$, $(2620, 2924)$, itd. Napisati funkciju "`BrojPrijateljskihParova`" koja kao parametar prima vektor cijelih brojeva a koja vraća kao rezultat koliko se u tom vektoru može naći parova prijateljskih brojeva (naravno, ukoliko takvih parova nema, rezultat će biti 0). Pri tome se, radi jednostavnosti, pretpostavlja da ulazni vektor ne sadrži duplikate, tj. elemente koji se ponavljaju (prisustvo duplikata može osjetno otežati rješavanje zadatka, s obzirom da će bez vođenja računa o tome da li je neki element duplikat ili ne često biti nađeno daleko više parova nego što ih zaista ima). Također treba napisati i glavni program kojem se za slijed od n brojeva koji se unose s tastature (pri čemu se n također unosi sa tastature) ispisuje koliko u tom slijedu ima prijateljskih parova brojeva. Dijalog između korisnika i programa treba izgledati poput sljedećeg:

```
Koliko ima brojeva: 8
Unesite brojeve: 5 284 2620 377 2924 49 220 3522
Broj prijateljskih parova: 2
```

Rješenje:

S obzirom da će trebati nalaziti sume djelilaca za dva broja koji čine potencijalni par, dupliranje koda možemo izbjeći tako što ćemo prvo napisati pomoćnu funkciju koja računa sumu djelilaca nekog broja, ne računajući njega samog. To ćemo učiniti tako što ćemo u petlji ispitivati sve brojeve od 1 do tog broja osim njega samog, i ukoliko je ispitivani broj djelilac tog broja, nadodaćemo ga na sumu (koja je inicijalno postavljena na nulu). To daje sljedeću veoma jednostavnu funkciju:

```
int SumaDjelilaca(int n) {
  int suma = 0;
  for(int i = 1; i < n; i++)
    if(n % i == 0) suma += i;
  return suma;
}
```

Sada bi se funkcija "`BrojPrijateljskihParova`" mogla napisati ovako. U dvije ugniježdene for petlje ispitujemo sve parove elemenata vektora v oblika $\{v_i, v_j\}$ da li možda čine prijateljski par brojeva, pri čemu je $j > i$. Ograničenje $j > i$ je potrebno iz dva razloga. Prvo, time se sprečava da i i j budu jednaki, odnosno sprečava se da se neki element vektora uparuje sa samim sobom (takav par koji obrazuju dva ista elementa bio bi proglašen za prijateljski, mada je jasno da kad kažemo "par elemenata" mislimo na dva različita elementa, tj. dva elementa na različitim pozicijama). Drugo, uvjet $\{v_i, v_j\}$ garantira da nećemo analizirati dva para koji se sastoje od istih elemenata. Na primjer, kada budemo testirali par $\{v_3, v_8\}$, nećemo kasnije analizirati par $\{v_8, v_3\}$ koji je obrazovan od istih elemenata, odnosno to je faktički isti par (u suprotnom bismo na kraju zaključili da ima dva puta više parova prijateljskih brojeva nego što ih zaista ima, jer bismo svaki par ispitivali dvaput). Za svaki par oblika $\{v_i, v_j\}$ testiraćemo da li je suma djelilaca broja v_i jednaka v_j i da li je suma djelilaca broja v_j jednaka v_i. Ukoliko jeste, našli smo jedan prijateljski par, tako da možemo uvećati brojač prijateljskih parova za 1 (naravno, ovaj brojač je na početku inicijaliziran na nulu). Sve u svemu, najneposrednija izvedba bila bi sljedeća (po cijenu da budemo dosadni, parametar funkcije bi bilo bolje deklarirati kao "`const std::vector<double> &v`" umjesto kao "`std::vector<double> v`"):

```
int BrojPrijateljskihParova(std::vector<int> v) {
  int brojac = 0;
  for(int i = 0; i < v.size(); i++)
    for(int j = i + 1; j < v.size(); j++)
      if(SumaDjelilaca(v[i]) == v[j] && SumaDjelilaca(v[j]) == v[i]) brojac++;
  }
  return brojac;
}
```

Nažalost, ova izvedba je, zbog mnogo razloga, daleko efikasnija nego što bi mogla da bude. Na prvom mjestu, unutar unutrašnje petlje, u pozivu "`SumaDjelilaca(v[i])`" stalno se računa suma djelilaca jednog te istog elementa "`v[i]`" (s obzirom da se "`i`" ne mijenja u unutrašnjoj petlji). Daleko bolje bi bilo tu sumu izračunati prije ulaska u unutrašnju petlju i sačuvati je u nekoj

pomoćnoj promjenljivoj, koju bismo samo koristili unutar tijela unutrašnje petlje. Time znatno smanjujemo broj izračunavanja jedne te iste vrijednosti. To bi izgledalo recimo ovako:

```
int BrojPrijateljskihParova(std::vector<int> v) {
  int brojac = 0;
  for(int i = 0; i < v.size(); i++) {
    int suma_djelilaca_vi = SumaDjelilaca(v[i]);
    for(int j = i + 1; j < v.size(); j++)
      if(suma_djelilaca_vi == v[j] && SumaDjelilaca(v[j]) == v[i]) brojac++;
  }
  return brojac;
}
```

Interesantno je da je pomoću jednog jednostavnog trika moguće izbjeći if naredbu u ovoj funkciji. Naime, cijelu if naredbu moguće je zamijeniti naredbom

```
brojac += suma_djelilaca_vi == v[j] && SumaDjelilaca(v[j]) == v[i];
```

Trik je zasnovan na automatskoj pretvorbi logičkih vrijednosti u brojčane. Rezultat uvjeta koji se nalazi iza operatora "`+=`" nadodaje se na brojčanu promjenljivu. S obzirom da je to korištenje logičke vrijednosti u kontekstu u kojem se očekuje brojčana vrijednost, "tačno" se interpretira kao 1, a "netačno" kao 0. Dakle, ukoliko je uvjet tačan, brojač će se povećati za 1, u suprotnom neće.

S obzirom da je računanje sume djelilaca jednostavan posao, mogli smo izbjeći pisanje pomoćne funkcije "`SumaDjelilaca`", po cijenu minornog dupliranja koda. To bi se moglo izvesti recimo ovako:

```
int BrojPrijateljskihParova(std::vector<int> v) {
  int brojac = 0;
  for(int i = 0; i < v.size(); i++) {
    int suma_djelilaca_vi = 0;
    for(int k = 1; k < v[i]; k++)
      if(v[i] % k == 0) suma_djelilaca_vi += k;
    for(int j = i + 1; j < v.size(); j++) {
      int suma_djelilaca_vj = 0;
      for(int k = 1; k < v[j]; k++)
        if(v[j] % k == 0) suma_djelilaca_vj += k;
      if(suma_djelilaca_vi == v[j] && suma_djelilaca_vj == v[i]) brojac++;
    }
  }
  return brojac;
}
```

Primijetimo da smo morali preimenovati brojač petlji koje traži djelioce u "`k`" da izbjegnemo konflikt s imenima brojača koji se koriste za dvije ugniježdene petlje u kojima se ove petlje nalaze (tako da ovdje zapravo imamo trostruko ugniježdene petlje). Posljednju if naredbu u ovoj funkciji možemo izbjeći na potpuno isti način kao u prethodnoj verziji funkcije, koja koristi pomoćnu funkciju.

Ove izvedbe su još uvijek znatno sporije nego što bi moglo biti. Istina je da smo izbjegli stalno računanje sume djelioca elementa "`v[i]`" u unutrašnjoj petlji, ali se u njoj stalno računaju sume djelioca elemenata "`v[j]`", i to svaki put ispočetka pri svakom prolasku kroz vanjsku petlju. Na taj način, funkcija "`SumaDjelilaca`" se u unutrašnjoj petlji poziva onoliko puta koliko ukupno ima parova koji se testiraju, tj. $n(n-1)/2 \approx n^2/2$ puta. S aspekta vremena izvršavanja, daleko efikasnije rješenje je na početku u neki pomoćni vektor (nazvan recimo "`sume_djelilaca`") izračunati i smjestiti sume djelilaca svih elemenata iz vektora "`v`", a onda unutar ugniježdenih petlji koje testiraju sve parove samo koristiti prethodno izračunate vrijednosti suma djelilaca. Time se funkcija "`SumaDjelilaca`" ukupno poziva svega n puta, umjesto oko $n^2/2$ puta, što je velika vremenska ušteda. Time dolazimo do sljedeće izvedbe:

```
int BrojPrijateljskihParova(std::vector<int> v) {
  std::vector<int> sume_djelilaca;
  for(int element : v) sume_djelilaca.push_back(SumaDjelilaca(element));
  int brojac = 0;
  for(int i = 0; i < v.size(); i++)
    for(int j = i + 1; j < v.size(); j++)
      if(suma_djelilaca[i] == v[j] && suma_djelilaca[j] == v[i]) brojac++;
  return brojac;
}
```

Jedina eventualna mana ove izvedbe je potreba za kreiranjem pomoćnog vektora, čime se dodatno troši memorija (dakle, s aspekta utroška memorije, ovo rješenje je manje efikasno od prethodnih rješenja). Ovo je inače poznati primjer tzv. *trgovine između vremena i memorije* (engl. *time-space tradeoff*). Naime, pri razvoju algoritama veoma čest je slučaj da se izvođenje nekog postupka može ubrzati po cijenu dodatnog zauzeća memorije, odnosno da je moguće uštedjeti memoriju po cijenu usporavanja vremena izvršavanja postupka. Pri tome, postoje određene granice, odnosno raspon u kojem je trgovina moguća, odnosno moguće je dostići granicu u kojoj dalje ubrzanje nije moguće postići nikakvim dodatnim zauzimanjem memorije, odnosno granicu u kojoj dodatna ušteda memorije nije moguća nikakvim dodatnim usporavanjem vremena izvođenja.

S obzirom da se u prethodnom rješenju pomoćna funkcija "`SumaDjelilaca`" poziva na samo jednom mjestu, sasvim je praktična (mada manje čitljiva) i izvedba u kojoj se potpuno eliminira potreba za ovom pomoćnom funkcijom, poput sljedeće izvedbe:

```
int BrojPrijateljskihParova(std::vector<int> v) {
  std::vector<int> sume_djelilaca;
  for(int element : v) {
    int suma = 0;
    for(int k = 1; k < element; k++)
      if(element % k == 0) suma += k;
    sume_djelilaca.push_back(suma);
  }
  int brojac = 0;
  for(int i = 0; i < v.size(); i++)
    for(int j = i + 1; j < v.size(); j++)
      if(suma_djelilaca[i] == v[j] && suma_djelilaca[j] == v[i]) brojac++;
  return brojac;
}
```

Preostaje još samo da se napiše glavni program, što nakon svih prethodnih rješenja zaista ne bi trebao biti nikakav problem. On bi mogao izgledati recimo ovako:

```
#include <iostream>
#include <vector>

// Ubaciti ovdje definiciju funkcije "SumaDjelilaca" (ako je korištena kao pomoćna)

// Ubaciti ovdje definiciju funkcije "BrojPrijateljskihParova"

int main() {
  int n;
  std::cout << "Koliko ima brojeva: ";
  std::cin >> n;
  std::cout << "Unesite brojeve: ";
  std::vector<int> v(n);
  for(int i = 0; i < v.size(); i++) std::cin >> v[i];
  std::cout << "Broj prijateljskih parova: " << BrojPrijateljskihParova(v);
  return 0;
}
```

11. Za neki niz brojeva $a_1, a_2, \ldots, a_n$ kažemo da je *aritmetički niz* ukoliko je razlika ma koja dva susjedna elementa uvijek ista, a *geometrijski niz* ukoliko je količnik ma koja dva susjedna elementa uvijek isti. Na primjer, nizovi 2, 5, 8, 11, 14, 17 i 29, 25, 21, 17, 13, 9 su aritmetički nizovi, dok su 2, 6, 18, 54, 162 i 40, 20, 10, 5 geometrijski nizovi. Napisati funkciju "`OdrediVrstuNiza`" koja kao parametar prima vektor cijelih brojeva, a koja vraća kao rezultat jednu od vrijednosti "`Aritmeticki`", "`Geometrijski`" ili "`Nespecificirani`", ovisno od toga da li elementi tog vektora obrazuju aritmetički niz, geometrijski niz, ili ništa od tog dvoje. Pri tome su "`Aritmeticki`", "`Geometrijski`" ili "`Nespecificirani`" pobrojane konstante pobrojanog tipa "`VrstaNiza`" s ograničenim vidokrugom. Potrebno je napisati i kratki testni program koji testira napisanu funkciju na nizu brojeva koji se unose putem tastature.

Rješenje:

Na prvom mjestu, potrebno je deklarirati pobrojani tip "`VrstaNiza`", što možemo izvesti ovako:

```
enum class VrstaNiza {Aritmeticki, Geometrijski, Nespecificirani};
```

Tražena funkcija zasniva se na manje-više rutinskoj primjeni testiranja da li su zadovoljeni svi uvjeti iz neke grupe uvjeta. Međutim, iako se radi o načelno laganom zadatku. treba obratiti pažnju

na neke moguće zamke u koje se lako može upasti ukoliko se radi brzopleto i ne razmisli dobro o svemu. Na prvom mjestu, prilikom testiranja da li je niz geometrijski, moglo bi doći do dijeljenja nulom. U tom slučaju, jasno je da niz nije geometrijski (pri čemu dijeljenje ne smijemo ni pokušati ukoliko bi to bilo dijeljenje nulom). Drugo, treba obratiti pažnju da iako su po pretpostavci zadatka svi elementi niza cijeli brojevi, količnik za slučaj geometrijskog niza ne mora biti cijeli broj, tako da moramo voditi računa da količnik ne bude izračunat cjelobrojnim dijeljenjem (recimo, niz 16, 24, 36, 54, 81 je geometrijski niz cijelih brojeva s količnikom 1.5). Treće, nizovi koji imaju manje od dva elementa sasvim očito ne mogu biti ni geometrijski ni aritmetički (ne provjeri li se ovo na samom početku, možemo imati pristup nelegalnom indeksu, što je nedefinirano ponašanje). Najbolje je, kad testiramo da li je niz aritmetički ili geometrijski, odmah na početku na osnovu prva dva člana izračunati kakva bi trebala biti razlika odnosno količnik kada bi niz bio aritmetički odnosno geometrijski, pa onda dalja testiranja raditi na osnovu toga (time se znatno pojednostavljuje i ubrzava testiranje unutar petlji). U skladu s tim, moguća izvedba tražene funkcije izgleda ovako (pri čemu bi parametar funkcije bilo bolje deklarirati kao "`const std::vector<int> &v`", što je već postala uobičajena napomena):

```
VrstaNiza OdrediVrstuNiza(std::vector<int> v) {
  if(v.size() < 2) return VrstaNiza::Nespecificirani;
  bool aritmeticki = true;
  int razlika = v[1] - v[0];
  for(int i = 2; i < v.size() && aritmeticki; i++)
    if(v[i] - v[i - 1] != razlika) aritmeticki = false;
  if(aritmeticki) return VrstaNiza::Aritmeticki;
  if(v[0] == 0) return VrstaNiza::Nespecificirani;
  double kolicnik = double(v[1]) / v[0];
  for(int i = 2; i < v.size(); i++)
    if(v[i - 1] == 0 || double(v[i]) / v[i - 1] != kolicnik)
      return VrstaNiza::Nespecificirani;
  return VrstaNiza::Geometrijski;
}
```

Ovdje treba ukazati pažnju na nekoliko detalja. Prvo, ukoliko su svi elementi niza isti, taj niz može biti i aritmetički (s razlikom 0) i geometrijski (s količnikom 1). S obzirom da u postavci zadatka nije specificirano kako treba tretirati ovaj slučaj, u ovoj izvedbi je usvojena konvencija da se takav niz tretira kao aritmetički. Drugo, pri testiranju da li je niz aritmetički, ukoliko naiđemo na situaciju koja jasno ukazuje da niz nije aritmetički, još uvijek ne znamo da li je možda geometrijski, ili nije nijedna od te dvije vrste, te moramo preći na dalja testiranja. S druge strane, pri testiranju da li je niz geometrijski, koje dolazi kasnije, ukoliko naiđemo na situaciju koja jasno ukazuje da niz nije geometrijski, možemo odmah napustiti funkciju i reći da niz nije ni aritmetički ni geometrijski, jer smo prethodno vidjeli da nije aritmetički, a treće mogućnosti nema. Konačno, obratimo pažnju na složeni uvjet "`v[i - 1] == 0 || double(v[i]) / v[i - 1] != kolicnik`". U ovom izrazu, zahvaljujući poduvjetu "`v[i - 1] == 0`" i specifičnom ponašanju operatora "`||`", nikada ne može doći do dijeljenja nulom. Naime, u izrazima oblika "x `||` y", prvo se računa "x", i ukoliko je "x" tačan, cijeli izraz se proglašava za tačan, bez da se "y" uopće pokušava izračunati (ovo je još jedan primjer kratkospojnog izračunavnja). Tek ukoliko je "x" netačan, računa se i "y", te od njegove tačnosti zavisi tačnost cijelog izraza. U ovom konkretnom primjeru, ukoliko je podizraz "`v[i - 1] == 0`" tačan, čitav izraz se proglašava tačnim i napuštamo funkciju, bez da se podizraz "`double(v[i]) / v[i - 1] != kolicnik`", koji bi proizveo dijeljenje nulom u slučaju kada je "`v[i - 1] == 0`" tačno, uopće pokuša izračunati.

Kad već govorimo o dijeljenju nulom, treba znati da se cjelobrojno dijeljenje nulom (tj. dijeljenje u kojem su oba operanda cjelobrojnog tipa) tretira kao *nedefinirano ponašanje*, i moguće je da se desi bilo šta (krah programa je tipičan scenario). S druge strane, realno dijeljenje nulom (tj. dijeljenje u kojem je makar jedan operand realnog tipa) ne mora biti nedefinirano ponašanje. Tu situacija zavisi od kompajlera koji se koristi. Naime, ukoliko kompajler koji koristimo ne podržava IEEE 754 standard za rad s realnim brojevima, realno dijeljenje nulom će također biti nedefinirano ponašanje. S druge strane, ukoliko korišteni kompajler podržava IEEE 754 standard (a to su praktično svi današnji kompajleri), realno dijeljenje nulom nije nedefinirano ponašanje, nego daje kao rezultat specijalan objekat "∞", za koji se smatra da je realnog tipa. Više o ovome može se naći u dodatku "Šokantna priča o realnoj aritmetici".

Ostaje još da napišemo testni program. Ovdje je jedini problem ukoliko želimo lijepo da ispišemo rezultat testiranja, jer promjenljive koje su pobrojanog tipa s ograničenim vidokrugom ne možemo ispisivati na ekran (obične promjenljive pobrojanog tipa možemo, ali pri tome dolazi do neželjene

pretvorbe pobrojane vrijednosti u brojčanu, odnosno ispis neće biti onakav kakav bismo željeli). Jedna mogućnost je da pomoću if naredbe oblikujemo ispis u skladu s željama, odnosno da dobijemo prihvatljiviji ispis. To dovodi do rješenja poput sljedećeg:

```
#include <iostream>
#include <vector>

enum class VrstaNiza {Aritmeticki, Geometrijski, Nespecificirani};

// Ubaciti ovdje definiciju funkcije "OdrediVrstuNiza"

int main() {
  int n;
  std::cout << "Broj elemenata: ";
  std::cin >> n;
  std::cout << "Elementi: ";
  std::vector<int> v(n);
  for(int i = 0; i < n; i++) std::cin >> v[i];
  VrstaNiza vrsta = OdrediVrstuNiza(v);
  std::cout << "Niz je ";
  if(vrsta == VrstaNiza::Aritmeticki) std::cout << "aritmeticki";
  else if(vrsta == VrstaNiza::Geometrijski) std::cout << "geometrijski";
  else std::cout << "nespecificirani";
  return 0;
}
```

Mada ovakvo rješenje radi, njegova neelegancija je očita, i pogotovo bi došla do izražaja u slučaju da smo imali veći broj pobrojanih vrijednosti, a ne samo tri. Elegantnije rješenje traži jednu sitnicu koja se obrađuje tek na Predavanju 3, ali ćemo je ipak ovdje spomenuti. Naime, ukoliko imamo neki pokazivač na znakove koji pokazuje na prvi znak nekog teksta (niza znakova između navodnika), pri pokušaju ispisa tog pokazivača na izlazni tok neće se ispisati adresa (kao što bi se desilo za druge vrste pokazivača), nego upravo tekst na čiji prvi znak pokazuje pokazivač. To omogućava da napravimo niz pokazivača na znakove koji ćemo inicijalizirati da pokazuju na prikladne tekstualne reprezentacije pobrojanih vrijednosti. Zatim ćemo pobrojanu vrijednost konvertiranu u cjelobrojnu šifru koristiti kao indeks u takvom nizu pokazivača, pri čemu ćemo dobijeni pokazivač poslati na izlazni tok. To će dovesti do ispisa odgovarajućeg teksta. Takvo rješenje izgledalo bi ovako:

```
#include <iostream>
#include <vector>

enum class VrstaNiza {Aritmeticki, Geometrijski, Nespecificirani};

// Ubaciti ovdje definiciju funkcije "OdrediVrstuNiza"

int main() {
  int n;
  std::cout << "Broj elemenata: ";
  std::cin >> n;
  std::cout << "Unesite elemente niza: ";
  std::vector<int> v(n);
  for(int i = 0; i < n; i++) std::cin >> v[i];
  const char *tekstovi[3]{"Aritmeticki", "Geometrijski", "Nespecificirani"};
  std::cout << "Niz je " << tekstovi[int(OdrediVrstuNiza(v))];
  return 0;
}
```

Jasno je da se ovakvo rješenje može lako generalizirati na proizvoljan broj pobrojanih vrijednosti.

12. Napisati funkciju koja prihvata dva cijela broja (tipa "`int`") p i q kao parametre, i koja vraća kao rezultat "tačno" ukoliko p i q sadrže makar jednu istu cifru, a u suprotnom vraća rezultat "netačno". Na primjer, ukoliko je $p = 35148$ i $q = 7386$, funkcija treba da vrati "tačno", jer p i q imaju cifre 3 i 8 kao zajedničke, dok za $p = 3715$ i $q = 82064$ funkcija treba da vrati "netačno", jer p i q nemaju ni jednu zajedničku cifru. Također napisati i kratki demonstracioni program koji ilustrira napisanu funkciju na dva broja koji se unose putem tastature.

Rješenje:

Ima jako mnogo načina da se riješi ovaj zadatak, pa ćemo ih mnogo i pokazati, jer se iz svakog od prikazanih načina može mnogo naučiti (kao i obično, prikaz različitih načina za rješavanje istog problema otvara vidike). Uglavnom većina načina svodi se na ideju "za svaku cifru prvog broja,

provjeriti da li se ta cifra nalazi i u drugom broju". Prvo rješenje je direktna implementacija te ideje, s dvije ugniježdene petlje, i koja ne troši nikakvu dodatnu memoriju (nizove, itd.). Posljednja cifra se, naravno, može dobiti kao apsolutna vrijednost ostatka pri dijeljenju s 10, nakon čega dijeljenjem s 10 odbacujemo posljednju cifru i ponavljamo postupak sve dok ima cifara:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  do {
    int q_kopija = q, cifra_p = std::abs(p % 10);
    do {
      if(std::abs(q_kopija % 10) == cifra_p) return true;
      q_kopija /= 10;
    } while(q_kopija != 0);
    p /= 10;
  } while(p != 0);
  return false;
}
```

Ovdje je upotrijebljena do-while petlja zbog mogućnosti da p ili q budu 0. S for ili while petljom za takav broj ne bi bila izvdojena ni jedna cifra (za nulu petlja ne bi uopće ni započela), pa se ne bi prijavilo da recimo brojevi 0 i 3502 imaju zajedničku cifru (0). Do-while petlju bismo mogli izbjeći ukoliko bismo $p = 0$ i $q = 0$ tretirali kao specijalne slučajeve, što u ovoj izvedbi nije posve lako. Recimo, mogli bismo postupiti ovako:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  if(p == 0) {
    for(; q != 0; q /= 10)
      if(std::abs(q % 10) == 0) return true;
    return false;
  }
  for(; p != 0; p /= 10) {
    int q_kopija = q, cifra_p = std::abs(p % 10);
    if(q == 0 && cifra_p == 0) return true;
    for(; q_kopija != 0; q /= 10)
      if(std::abs(q_kopija % 10) == cifra_p) return true;
  }
  return false;
}
```

Mana ovako prikazanih rješenja je što se isti proces izdvajanja cifara broja q ponavlja za svaku cifru broja p. Bolje je prvo izvdojiti sve cifre broja q u neki kontejner, recimo vektor, a zatim za svaku cifru broja p pretraživati taj vektor (ne smeta što se cifre mogu ponavljati). Nedostatak tog rješenja je dodatno zauzeće memorije za vektor. Uz do-while petlje, takvo rješenje moglo bi se izvesti ovako:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  std::vector<int> cifre_q;
  do {
    cifre_q.push_back(std::abs(q % 10));
    q /= 10;
  } while(q != 0);
  do {
    int cifra_p = std::abs(p % 10);
    for(int cifra_q : cifre_q)
      if(cifra_q == cifra_p) return true;
    p /= 10;
  } while(p != 0);
  return false;
}
```

Nakon Predavanja 5 ćemo vidjeti da se umjesto unutrašnje rasponske for petlje može koristiti i poziv funkcije "`find`" iz biblioteke sa zaglavljem "`algorithm`", samo što to neće niti bitno skratiti rješenje, niti će ga ubrzati (ipak, interesantno je vidjeti i takvo rješenje).

Prethodno rješenje može se također prepraviti da radi s for ili while petljama umjesto do-while petlji, ali se $p = 0$ i $q = 0$ moraju tretirati kao specijalni slučajevi. To je lakše izvesti za q (dovoljno je samo dodati jednu nulu u vektor ako je $q = 0$) nego za p. Jedna moguća izvedba zasnovana na ovoj ideji, izgledala bi ovako:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  std::vector<int> cifre_q;
  if(q == 0) cifre_q.push_back(0);
  for(; q != 0; q /= 10) cifre_q.push_back(std::abs(q % 10));
  if(p == 0) {
    for(int cifra_q : cifre_q)
      if(cifra_q == 0) return true;
    return false;
  }
  for(; p != 0; p /= 10) {
    int cifra_p = std::abs(p % 10);
    for(int cifra_q : cifre_q)
      if(cifra_q == cifra_p) return true;
  }
  return false;
}
```

Kasnije ćemo vidjeti da se obje unutrašnje rasponske for petlje mogu također zamijeniti pozivom bibliotečke funkcije "`find`", čime se može dobiti rješenje koje je donekle elegantnije, mada nije bitno kraće od ovog rješenja, niti je išta brže od njega.

Kao što smo izdvojili cifre broja q u vektor, možemo isto to uraditi i s brojem p, a onda samo porediti imaju li dva vektora zajednički element. Uz pomoć do-while petlje, to bi moglo izgledati ovako:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  std::vector<int> cifre_p, cifre_q;
  do {
    cifre_p.push_back(std::abs(p % 10));
    p /= 10;
  } while(p != 0);
  do {
    cifre_q.push_back(std::abs(q % 10));
    q /= 10;
  } while(q != 0);
  for(int cifra_p : cifre_p)
    for(int cifra_q : cifre_q)
      if(cifra_p == cifra_q) return true;
  return false;
}
```

Ovdje je lako napraviti i rješenje zasnovano na while ili for petlji, jer se oba specijalna slučaja $p = 0$ i $q = 0$ lagano tretiraju (samo treba dodati nulu u vektor). Takvo rješenje izgledalo bi ovako:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  std::vector<int> cifre_p, cifre_q;
  if(p == 0) cifre_p.push_back(0);
  for(; p != 0; p /= 10) cifre_q.push_back(std::abs(q % 10));
  if(q == 0) cifre_q.push_back(0);
  for(; q != 0; q /= 10) cifre_q.push_back(std::abs(q % 10));
  for(int cifra_p : cifre_p)
    for(int cifra_q : cifre_q)
      if(cifra_p == cifra_q) return true;
  return false;
}
```

Unutrašnje rasponske for petlje se i u ovim rješenjima mogu zamijeniti pozivom bibliotečke funkcije "`find`", o čemu ćemo govoriti nakon Predavanja 5. Međutim, nedostatak opisanih rješenja u kojima se cifre oba broja p i q izdvajaju u vektor je, pored toga što se koriste dva pomoćna vektora, to što se praktično isti kôd ponavlja dva puta, jedanput za izdvajanje cifara broja p, a drugi put za izdvajanje cifara broja q. To navodi na ideju da bi se mogla koristiti pomoćna funkcija za izdvajanje cifara, koja bi uz upotrebu do-while petlje mogla izgledati ovako:

```
std::vector<int> IzvdojiCifre(int n) {
  std::vector<int> v;
  do {
    v.push_back(std::abs(n % 10));
    n /= 10;
  } while(n != 0);
  return v;
}
```

Alternativno, uz izbjegavanje do-while petlje, ista funkcija mogla bi se napisati ovako:

```
std::vector<int> IzdvojiCifre(int n) {
  std::vector<int> v;
  if(n == 0) v.push_back(0);
  for(; n != 0; n /= 10) v.push_back(std::abs(n % 10));
  return v;
}
```

Uz ovu pomoćnu funkciju, tražena funkcija se može napisati zaista vrlo elegantno:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  for(int cifra_p : IzdvojiCifre(p))
    for(int cifra_q : IzdvojiCifre(q))
      if(cifra_o == cifra_q) return true;
  return false;
}
```

Moguće je i ovdje unutrašnju rasponsku for petlju zamijeniti pozivom bibliotečke funkcije "`find`", ali u ovom slučaju izvedba nije toliko elegantna kolika je moguća u ranijim rješenjima. Uglavnom, priču o tome ostavićemo za nakon Predavanja 5.

Nažalost, sva prethodno prikazana rješenja imaju zajednički nedostatak: znatno su sporija nego što bi mogla biti. Naime, u svim dosada prikazanim rješenjima javljaju se ugniježdene petlje (pri tome se kao alternativa unutrašnjim petljama može koristiti poziv bibliotečke funkcije "`find`", ali to ništa suštinski ne mijenja, jer i ona interno izvodi petlju). Stoga, ukoliko prvi broj ima m, a drugi n cifara, samo za te dvije ugniježdene petlje troši se vrijeme proporcionalno s mn, jer se vanjska petlja izvodi m puta, a unutrašnja n puta. Međutim, interesantno je da postoji rješenje koje troši vrijeme koje je proporcionalno svega s $m+n$ (umjesto s mn). Recimo, za $m = n = 10$ to daje ubrzanje od 5 puta, a za $m = n = 20$ ubrzanje je 10 puta. Ideja ovog rješenja je da uvedemo pomoćni niz od 10 logičkih elemenata koji predstavljaju "markere", po jedan za svaku cifru (konkretno, element govori da li je cifra "markirana" ili ne, pri čemu na početku nijedna cifra nije "markirana"). Svaki put kad u broju p izdvojimo novu cifru, "markiramo" tu cifru kao prisutnu. Nakon što izvdojimo sve cifre iz broja p, izdvajamo cifre iz broja q i provjeravamo da li je izdvojena cifra već "markirana". Ukoliko jeste, q sadrži istu cifru kao i p, te je postupak gotov. Međutim, ukoliko u broju q ne nađemo ni jednu "markiranu" cifru tada p i q nemaju zajedničkih cifara. Takva izvedba, uz korištenje do-while petlji, izgledala bi ovako:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  bool markirana [10]{};
  do {
    markirana[std::abs(p % 10)] = true;
    p /= 10;
  } while(p != 0);
  do {
    if(markirana[std::abs(q % 10)]) return true;
    q /= 10;
  } while(q != 0);
  return false;
}
```

Niz "`markirana`" inicijaliziran je praznim vitičastim zagradama, što zapravo sve njegove elemente inicijalizira na podrazumijevane vrijednosti (za tip "`bool`", podrazumijevana vrijednost je "`false`"). Nije teško prepraviti prethodno rješenje da umjesto do-while petlje koristi while ili for petlju. Recimo, jedno takvo rješenje izgledalo bi ovako:

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  bool markirana[10]{};
  if(p == 0) markirana[0] = true;
  for(; p != 0; p /= 10) markirana[std::abs(p % 10)] = true;
  if(q == 0 && ima_cifre[0]) return true;
  for(; p != 0; p /= 10)
    if(markirana[std::abs(q % 10)]) return true;
  return false;
}
```

Ubrzanje je ostvareno time što umjesto dvije ugniježdene petlje u ovim rješenjima imamo dvije posve odvojene petlje, jednu nakon druge. Naravno, umjesto običnog niza možemo koristiti i moderni niz, što je generalno čak i bolje, s obzirom da se upotreba običnih nizova ne preporučuje u C++ programima (korištenje vektora za tu svrhu ne bi bilo preporučljivo, jer je broj elemenata fiksan, tako da nam puna fleksibilnost vektora nije potrebna). Recimo, posljednje rješenje bi se, uz korištenje modernih nizova, moglo napisati recimo ovako (ne zaboravimo da korištenje modernih nizova traži uključenje biblioteke sa zaglavljem "`array`" u program):

```
bool DaLiSadrzeIstuCifru(int p, int q) {
  std::array<bool, 10> markirana{};
  if(p == 0) markirana[0] = true;
  for(; p != 0; p /= 10) markirana[std::abs(p % 10)] = true;;
  if(q == 0 && ima_cifre[0]) return true;
  for(; p != 0; p /= 10)
    if(markirana[std::abs(q % 10)]) return true;
  return false;
}
```

Ostalo je još da napišemo kratki demonstracioni program, što je u ovom zadatku jednostavno. Recimo, traženi demonstracioni program mogao bi izgledati ovako:

```
#include <iostream>
#include <vector>                // Samo ako su za realizaciju korišteni vektori
#include <array>                 // Samo ako su za realizaciju korišteni moderni nizovi

// Ubaciti ovdje definiciju funkcije "IzdvojiCifre" (ako je korištena kao pomoćna)

// Ubaciti ovdje definiciju funkcije "DaLiSadrzeIstuCifru"

int main() {
  int broj_1, broj_2;
  std::cout << "Unesite dva broja: ";
  std::cin >> broj_1 >> broj_2;
  std::cout << "Uneseni brojevi ";
  if(DaLiSadrzeIstuCifru(broj_1, broj_2)) std::cout << " sadrze istu cifru";
  else std::cout << "ne sadrze nijednu istu cifru";
  return 0;
}
```